

Độ sâu, dòng quang và mô hình nền hình học (Depth, Flow and Geometric Foundation Models)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng để quyết định thay hay không thay); F/28-dac-trung-va-ghep-cap (ghép cặp thừa hồng ở đâu); C/16/02-bieu-dien-3d-va-depth (tính không xác định được của tỉ lệ, bốn nguồn độ sâu); C/15/03-flow-tracking-va-rang-buoc (bài toán khẩu độ, Lucas-Kanade, kim tự tháp).

TÓM TẮT

Chương này hỏi một câu duy nhất: ba họ mô hình đang rất nóng — độ sâu đơn ảnh, dòng quang dày, và mô hình nền hình học — thêm được *thông tin* gì cho một hệ stereo-inertial vốn đã có tỉ lệ mét thật. Đọc xong bạn biết chính xác chỗ nào chúng thêm thông tin thật, chỗ nào chúng chỉ thêm phỏng đoán, và bạn tính được bằng số khi nào một tiên nghiệm dày sẽ đè bẹp phép đo thừa của bạn trong bài toán bình phương tối thiểu. Bạn cũng đọc được ba thiết kế đáng học nhất trong nhóm này: vòng lặp tra cứu của RAFT, phép chuẩn hoá theo tiêu cự của Metric3D, và cách DUST3R đổi biểu diễn đầu ra để xoá luôn bước ước lượng tư thế. Nếu chỉ nhớ một điều: những mô hình này không cho bạn phép đo mới, chúng cho bạn phỏng đoán dày — và chúng không bao giờ nói “tôi không biết”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tỉ lệ (scale)** — ba nghĩa khác nhau trong chương này — (a) *tỉ lệ mét* của một bản đồ SLAM: hệ số nhân biến toạ độ bản đồ thành mét, thứ mà stereo và gia tốc kế cố định được; (b) *tỉ lệ* trong “bất biến tỉ lệ và dịch”: một hệ số nhân tự do trên đầu ra của mạng độ sâu, khác nhau ở mỗi khung hình; (c) *đa tỉ lệ* của kim tự tháp ảnh: mức thu nhỏ. Ba nghĩa không liên quan gì nhau, và trộn chúng là nguồn hiểu nhầm lớn nhất của cả chủ đề này.
- **độ chênh lệch (disparity)** — với một cặp ảnh stereo đã nắn thẳng hàng, đó là hiệu toạ độ ngang của cùng một điểm giữa ảnh trái và ảnh phải, tính bằng pixel. Nó tỉ lệ nghịch với khoảng cách: $d = fb/Z$, trong đó f là tiêu cự tính bằng pixel, b là **đường cơ sở** (stereo baseline) tính bằng mét.
- **độ sâu nghịch đảo (inverse depth)** — đại lượng $1/Z$. Mạng độ sâu hiện đại hầu hết dự đoán đại lượng này chứ không dự đoán Z , vì nó bị chặn (điểm ở vô cực cho giá trị 0) và vì sai số của nó gần với sai số pixel hơn.
- **bản đồ điểm (pointmap)** — một tensor kích thước $H \times W \times 3$: mỗi pixel của ảnh được gán thẳng một điểm ba chiều (X, Y, Z) trong một hệ toạ độ nào đó. Khác với bản đồ độ sâu ở chỗ nó đã mang sẵn hệ toạ độ, nên không cần biết tiêu cự mới đọc ra được hình học.
- **khối chi phí (cost volume)** — bảng ghi mức giống nhau giữa mỗi vị trí ở ảnh này và mỗi vị trí ứng viên ở ảnh kia. Nó là dạng dữ liệu chung của mọi bài toán tương ứng: stereo, dòng quang, ghép cặp.
- **làm cong (warping)** — lấy ảnh hoặc bản đồ đặc trưng thứ hai rồi dịch từng pixel theo một trường chuyển động ước lượng, để nó nằm chồng lên ảnh thứ nhất. Sau khi làm cong, phần sai lệch còn lại chính là phần chưa ước lượng đúng.
- **ma trận thông tin** — nghịch đảo của ma trận hiệp phương sai. Trong một bài toán bình phương tối thiểu có trọng số, nó chính là trọng số: một phép đo có ma trận thông tin lớn kéo nghiệm về phía nó mạnh hơn. Cộng thêm phép đo là cộng thêm thông tin, và tổng thông tin quyết định nghiệm.

- **sai lệch có hệ thống (bias) so với nhiễu ngẫu nhiên** — nhiễu ngẫu nhiên triệt tiêu khi cộng nhiều phép đo; sai lệch có hệ thống thì không, vì mọi phép đo lệch cùng một hướng. Trong chương này, sai số của mạng độ sâu gần với loại thứ hai hơn — đó là lý do nó nguy hiểm.

CÂU HỎI MỞ ĐẦU

- Q1. Hệ của bạn là stereo-inertial, đã có tỉ lệ mét thật từ đường cơ sở và gia tốc kế. Vậy một mạng độ sâu đơn ảnh — vốn không có đơn vị nào cả — còn thêm được *thông tin* gì mà bạn chưa có?
- Q2. RAFT bỏ kim tự tháp ảnh nhưng vẫn dùng một kim tự tháp. Nó dựng kim tự tháp trên cái gì, và vì sao chính chỗ đặt kim tự tháp quyết định một vật nhỏ bay nhanh có sống sót hay không?
- Q3. DUST3R nhận hai ảnh chưa hiệu chỉnh và trả về hình học ba chiều mà không hề chạy bước ước lượng tư thế. Tư thế biến đi đâu, hay chính xác hơn, nó nằm ở chỗ nào trong đầu ra?
- Q4. Bạn nhét 5000 pixel độ sâu học được vào cùng bài toán tối ưu với 20 điểm stereo đo thật. Vì sao mẹo “đặt trọng số nhỏ cho độ sâu học được” không cứu được bạn, và phép tính nào cho thấy điều đó trong ba mươi giây?
- Q5. Vì sao một mô hình độ sâu dựa trên khuếch tán lại phá hỏng đúng cái harness chạy lặp mười lần mà bạn đã mất nhiều công để làm cho tắt định?

1.1 Đặt vấn đề

Đọc nhanh. Nếu chỉ có mười phút: đọc Hình 1.1 và mục *Cổng đánh giá* ở cuối. Kết luận của cả chương là ba họ mô hình này chưa nên thay khối nào trong *frontend* của bạn, nhưng hai ý tưởng thiết kế trong đó thì đáng mang đi ngay: tách phần tính một lần khỏi phần lặp, và mô hình hoá sai số tương quan bằng một biến phụ thay vì hạ trọng số.

Chương 28 thay bộ phát hiện và bộ ghép cặp trong *frontend*. Chương này đi xa hơn một bước: nó hỏi xem có nên thay chính *phép đo* bằng một phỏng đoán học được hay không. Ba họ mô hình đang đứng xếp hàng ngoài cửa hệ thống của bạn.

- **Độ sâu từ một ảnh** — đưa vào một khung hình, trả về một giá trị độ sâu cho mọi pixel. Không cần cặp stereo, không cần thị sai, không cần chuyển động.
- **Dòng quang dày** — đưa vào hai khung hình, trả về một vector dịch chuyển cho mọi pixel. Không cần bộ phát hiện điểm đặc trưng.
- **Mô hình nền hình học** — đưa vào một nhóm ảnh chưa hiệu chỉnh, trả về thẳng hình học ba chiều và tư thế camera. Không cần cả pipeline hình học cổ điển.

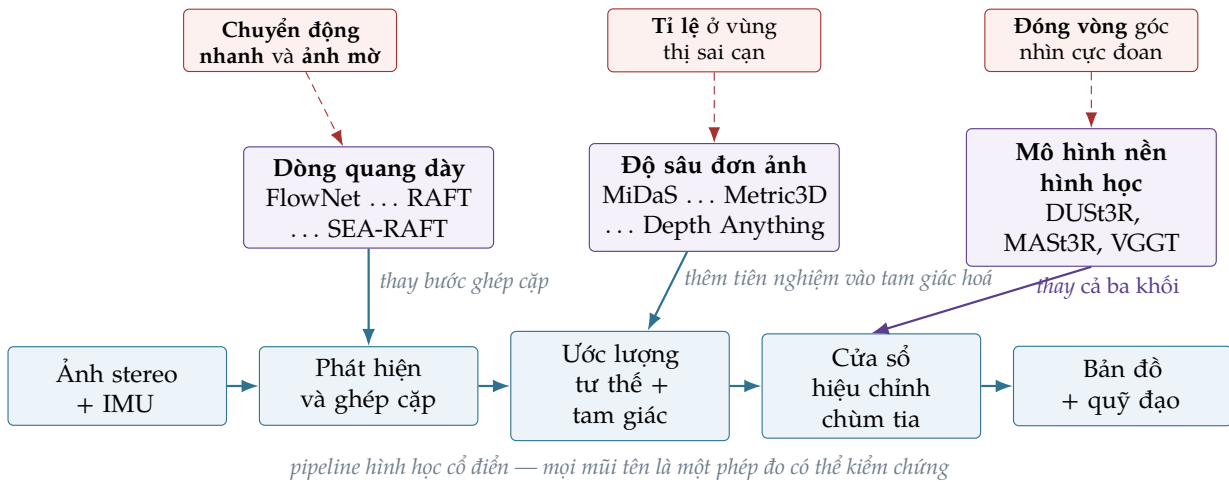
Ba lời chào hàng đó nghe rất khác nhau nhưng có chung một cấu trúc: chúng *đổi thưa lấy dày*. Hệ của bạn hiện sống bằng vài trăm phép đo mỗi khung hình, mỗi phép đo kèm một bài kiểm tra để loại điểm sai. Ba họ trên cho bạn hàng trăm nghìn giá trị mỗi khung hình, không kèm bài kiểm tra nào.

Và đây là chỗ phải nói thẳng ngay từ đầu, vì nó quyết định cách đọc cả chương. Bạn đang chạy một hệ *stereo-inertial*. Đường cơ sở stereo cho bạn tỉ lệ mét từ hình học; gia tốc kế cho bạn tỉ lệ mét từ vật lý. Phần lớn giá trị mà các bài báo độ sâu đơn ảnh quảng cáo — “lấy được hình học ba chiều từ một ảnh” — bạn đã có rồi, và có bằng phép đo chứ không bằng phỏng đoán. Vì vậy câu hỏi đúng không phải “mô hình nào chính xác nhất” mà là câu hỏi hẹp hơn nhiều: *ở đúng những pixel nào phép đo của tôi cạn kiệt, và ở đó một phỏng đoán học được có tốt hơn không có gì hay không*.

Học xong chương này bạn làm được gì mà trước đó không làm được? Bạn khoanh được bằng số vùng ảnh mà stereo của bạn đã hết thông tin. Bạn biết chi phí của từng họ mô hình *tỉ lệ với cái gì*, nên bạn loại được phương án sai cả bậc trước khi cài đặt, và bạn biết phép đo nào biến quan

hệ tỉ lệ ấy thành một con số dùng được cho phần cứng của mình. Và bạn tính được ngưỡng mà một tiên nghiệm dày bắt đầu lấn át phép đo thừa trong bộ tối ưu.

Hình 1.1 đặt ba họ mô hình cạnh nhau theo đúng chỗ chúng cắm vào một hệ SLAM, và theo nỗi đau mà mỗi họ hứa chữa.



Hình 1.1. Ba họ mô hình của chương này, đặt theo đúng khối mà chúng thay thế trong một hệ SLAM cổ điển, và theo nỗi đau mà mỗi họ hứa chữa (mũi tên đứt màu đỏ). Điểm cần chú ý là mức độ xâm lấn tăng dần từ trái sang phải: dòng quang chỉ thay một bước, độ sâu đơn ảnh chèn thêm một tiên nghiệm vào bước tam giác hoá, còn mô hình nền hình học nuốt luôn cả ba khối và trả về hình học đã dựng sẵn. Mức xâm lấn càng cao thì càng khó chẩn đoán khi hỏng, vì càng ít ranh giới để đặt điểm dừng.

TRỰC GIÁC

Hãy tìm một người bạn trong sân vận động đông người. Cách kim tự tháp ảnh: bạn lùi ra thật xa cho cả sân lọt vào tầm mắt, thấy một đám đông ở khán đài B, rồi tiến lại gần dần. Nếu bạn ấy đứng lẻ ở góc sân thì ở tầm nhìn xa bạn ấy chỉ là một chấm mờ, bạn đã đi nhầm về khán đài B, và mọi bước tiến lại gần chỉ tinh chỉnh quanh chỗ sai. Cách của RAFT khác hẳn: trước tiên lập một bảng ghi mức giống nhau giữa *mọi* cặp vị trí, rồi mỗi vòng lặp chỉ đọc vài ô quanh phỏng đoán hiện tại. Khi phỏng đoán còn xa đích, bạn đọc ở mức thô của *bảng* — nhưng cái bị làm thô là bảng, không phải sân, nên người đứng lẻ ở góc sân không hề biến mất. Toàn bộ chương này quay quanh đúng phân biệt đó: làm thô cái gì, và cái gì thì tuyệt đối không được làm thô.

1.2 Độ sâu từ một ảnh và vấn đề tỉ lệ (Monocular Depth and the Scale Problem)

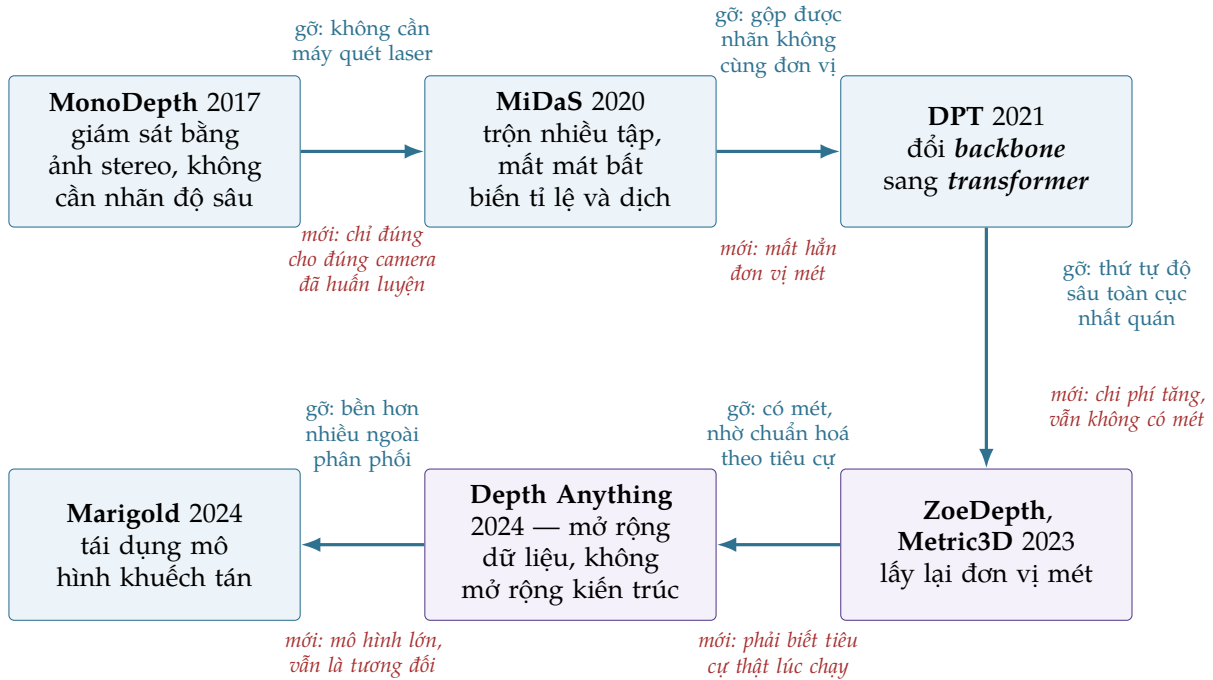
A. Bài toán gốc, và vì sao nó không phải bài toán khó mà là bài toán không giải được

Chương 16 đã chứng minh điều này bằng một hình vẽ, nên ở đây chỉ cần nhắc lại kết luận. Nhân toàn bộ cảnh với hệ số k và nhân luôn quãng đường camera với k thì mọi ảnh thu được giống hệt. Vì thế không lượng dữ liệu nào chữa được, chỉ **tiên nghiệm** (*prior*) mới chữa được.

↔ **tiền đề** C/16/02 — tính không xác định được của tỉ lệ đã được dựng đầy đủ ở đó; chương này chỉ dùng lại kết luận và đi tiếp vào hệ quả kỹ thuật.

Điều đó đặt ra ngay ranh giới đọc cả mục này: mọi con số độ sâu mà một mạng đơn ảnh trả về đều là *suy đoán có học*, không phải phép đo. Câu hỏi kỹ thuật thật sự là suy đoán ấy dựa vào tiên nghiệm gì, và tiên nghiệm ấy vỡ ở đâu.

Sáu thể hệ trong dòng này có thể đọc như sáu lần trả lời cho cùng một câu: *lấy giám sát ở đâu ra*. Hình 1.2 xếp chúng trên một trục, mỗi mũi tên ghi nút thắt được gỡ và vấn đề mới sinh ra.



Hình 1.2. Sáu bước của dòng độ sâu đơn ảnh, đọc theo câu hỏi “lấy giám sát ở đâu”. Đọc hàng trên từ trái sang phải rồi vòng xuống hàng dưới đi ngược lại, theo chiều mũi tên. Nhãn xanh là nút thắt được gỡ, nhãn đỏ nghiêng là vấn đề mới sinh ra. Chú ý hai ô tô tím: chúng là hai cách trả lời khác nhau cho cùng câu hỏi đơn vị mét — ZoeDepth gắn thêm một đầu ra riêng cho từng miền, còn Metric3D chuẩn hoá đầu vào để mọi ảnh trông như chụp bằng một camera duy nhất.

B. Từ MonoDepth tới Depth Anything, đọc theo nguồn giám sát

MonoDepth [1] là bước đầu tiên đáng nhớ vì nó xoá bỏ nhu cầu có nhãn độ sâu. Mạng nhận ảnh trái và dự đoán một trường độ chênh lệch; ta dùng trường đó để làm cong ảnh phải về vị trí ảnh trái; sai lệch quang trắc giữa hai ảnh chính là **hàm mất mát** (*loss function*). Thêm một số hạng buộc độ chênh lệch dự đoán từ ảnh trái và từ ảnh phải phải nhất quán với nhau. Vì lúc huấn luyện đường cơ sở đã biết, đầu ra thực chất có đơn vị mét — nhưng chỉ đúng cho đúng cấu hình camera đó. Monodepth2 sau này sửa hai chỗ hỏng cụ thể: dùng cực tiểu thay vì trung bình của sai lệch quang trắc qua các khung nguồn để chịu được vùng bị che, và tự động che các pixel không đối giữa hai khung để loại vật thể chuyển động cùng tốc độ với camera [2].

Chỗ hỏng thứ hai đáng dừng lại một nhịp, vì nó là ví dụ đẹp về việc một hàm mất mát tự tạo ra một nghiệm sai. Xét một chiếc xe chạy phía trước, cùng vận tốc với camera. Giữa hai khung hình liên tiếp, mọi pixel của chiếc xe đó *không dịch chuyển*. Bây giờ hỏi: giá trị độ sâu nào làm cho phép làm cong không dịch chuyển pixel? Đáp án là độ sâu vô cực. Vậy mạng có thể đạt sai lệch quang trắc bằng đúng không trên cả chiếc xe, chỉ bằng cách khai báo nó ở vô cùng xa. Triệu chứng đặc trưng là những cái lỗ đen ngòm hình chiếc xe trong bản đồ độ sâu. Không có gì sai trong quá trình tối ưu; hàm mất mát đúng là được cực tiểu hoá. Cái sai nằm ở giả định cảnh tĩnh, và cách chữa của Monodepth2 — che các pixel không đối giữa hai khung — là ép giả định đó thành một điều kiện lọc chứ không phải một hy vọng.

MiDaS [3] giải một bài toán khác hẳn: làm sao huấn luyện trên mười tập dữ liệu mà nhãn của chúng không cùng đơn vị. Có tập là quét laser ra mét, có tập là ảnh 3D từ phim với đường cơ sở không ai biết, có tập là kết quả dựng lại từ ảnh nên chỉ đúng sai khác một hệ số. Lời giải là bỏ luôn tham vọng về đơn vị. Trước khi tính sai lệch, ta căn dự đoán về nhãn bằng một phép biến

đối bậc nhất tối ưu: tìm s và t làm cực tiểu $\sum_i (s \hat{d}_i + t - d_i)^2$, rồi mới đo phần dư. Mạng vì thế chỉ phải học *hình dạng* của trường độ sâu nghịch đảo, không phải giá trị tuyệt đối của nó. Kết quả là khả năng khái quát vượt trội, và một cái giá rất cụt ngủn mà mục sau sẽ khai thác kỹ.

DPT [4] giữ nguyên công thức dữ liệu và hàm mất mát của MiDaS, chỉ đổi *backbone* sang *transformer* với một bộ giải mã dày. Lý do nó ăn thua nằm ở **trường tiếp nhận** (*receptive field*): trong một mạng tích chập, tầng có tầm nhìn toàn ảnh cũng là tầng có độ phân giải thấp nhất, nên thông tin toàn cục và chi tiết biến không bao giờ gặp nhau ở cùng một tầng. **Attention** có tầm nhìn toàn ảnh ngay từ tầng đầu, nên mô hình giữ được thứ tự độ sâu nhất quán trên toàn khung hình mà không phải hy sinh biên. Với người làm SLAM, đây là điểm đáng chú ý riêng: *thứ tự độ sâu toàn cục* chính là thứ mà mọi bộ ước lượng cục bộ đều thiếu.

ZoeDepth [5] và **Metric3D** [6] đi lấy lại đơn vị mét bằng hai đường khác nhau. ZoeDepth giữ mạng độ sâu tương đối làm phần thân, rồi gắn thêm một đầu ra dự đoán các khoảng độ sâu có đơn vị, tinh chỉnh riêng cho từng miền, cộng một bộ định tuyến chọn đầu ra theo ảnh. Nó dùng được khi ảnh thử nghiệm rơi vào đúng một trong các miền đã huấn luyện. Metric3D làm điều thú vị hơn nhiều, và mục sau dành riêng cho nó.

Depth Anything [7] là một bài học về dữ liệu chứ không phải về kiến trúc. Một mạng thầy được huấn luyện trên phần dữ liệu có nhãn, rồi gán nhãn giả cho hàng chục triệu ảnh không nhãn; mạng trò học trên nhãn giả đó nhưng dưới nhiều loạn mạnh, cộng một số hạng buộc đặc trưng của nó bám theo một mô hình nền thị giác đã đóng băng. Ý tưởng cốt lõi: chỉ chép lại nhãn giả thì trò không giỏi hơn thầy, nên phải làm bài toán *khó hơn* cho trò. Bản v2 [8] đổi một chi tiết có sức nặng bất ngờ: thầy được huấn luyện hoàn toàn trên ảnh tổng hợp. Lý do rất rõ ràng khi nói ra — nhãn độ sâu từ cảm biến thật luôn có lỗi, có nhiễu, và sai bét ở vật trong suốt hoặc phản chiếu, còn nhãn tổng hợp thì hoàn hảo tuyệt đối. Khoảng cách miền giữa ảnh tổng hợp và ảnh thật sau đó được lấp bằng chính bước gán nhãn giả trên ảnh thật. Có một bản v3 xuất hiện trong giai đoạn 2025–2026, theo mô tả công khai thì nó mở rộng cùng công thức sang dự đoán hình học từ số lượng khung hình bất kỳ; tôi chưa tự kiểm chứng chi tiết, nên chỉ nhắc tên và ý tưởng.

Marigold [9] tái dụng một mô hình khuếch tán sinh ảnh đã huấn luyện sẵn: giữ nguyên bộ mã hoá tiềm ẩn, tinh chỉnh phần khử nhiễu để nó sinh ra *bản đồ độ sâu* với điều kiện là ảnh đầu vào. Điều đáng kinh ngạc là nó chỉ cần một lượng ảnh tổng hợp rất nhỏ và một ngân sách huấn luyện rất khiêm tốn so với việc huấn luyện từ đầu, vì tiên nghiệm về cấu trúc cảnh đã nằm sẵn trong mô hình sinh. Biên rất sắc, khái quát rất tốt. Nhưng với hệ của bạn thì hai tính chất của nó là án tử: suy luận cần nhiều bước khử nhiễu, và *nó ngẫu nhiên* — cùng một ảnh, hai lần chạy cho hai bản đồ độ sâu khác nhau, nên người ta phải chạy nhiều lần rồi lấy trung bình. Ta sẽ quay lại chỗ này ở mục nói về chẩn đoán.

Cuối cùng, **Align3R** là một hướng đáng theo dõi vì nó tấn công đúng điểm yếu chí mạng của cả họ này: sự không nhất quán giữa các khung hình. Ý tưởng là kết hợp một mô hình độ sâu đơn ảnh với một mô hình bản đồ điểm hai ảnh, rồi tối ưu chung để tìm tỉ lệ và độ dịch của từng khung sao cho các khung khớp nhau trong không gian ba chiều. Đây là công trình rất mới, tôi không nắm chắc chi tiết nên chỉ nêu ý tưởng và không mô tả kết quả.

Bảng 1.1. Sáu họ độ sâu đơn ảnh. Cột giá trị nhất là cột cuối: mỗi họ mượn tiền nghiệm từ một nguồn khác nhau, nên nó hông đúng khi nguồn đó cạn. Cột đơn vị cho biết đầu ra có đọc thẳng ra mét được hay không.

Họ	Đơn vị	Cơ chế và giả định	Hông khi nào
MonoDepth	mét, riêng cho một camera	Giám sát bằng sai lệch quang trắc giữa hai ảnh stereo; giả định cảnh tĩnh và độ sáng không đổi	Đổi camera hoặc đổi đường cơ sở; vật chuyển động cùng tốc độ camera; vùng bị che ở biên
MiDaS, DPT	không có	Mất mát bất biến tỉ lệ và dịch trên độ sâu nghịch đảo, trộn nhiều tập dữ liệu	Mọi chỗ cần đơn vị; và tỉ lệ với độ dịch <i>khác nhau ở mỗi khung hình</i>
ZoeDepth	mét, theo miền	Thân tương đối cộng một đầu ra có đơn vị, tinh chỉnh riêng từng miền, có bộ định tuyến	Ảnh rơi ra ngoài mọi miền đã huấn luyện; bộ định tuyến chọn nhầm miền
Metric3D	mét	Chuẩn hoá ảnh về một camera chuẩn theo tiêu cự, dự đoán, rồi chuyển ngược lại	Tiêu cự khai báo sai; ống kính méo mạnh hoặc mất cá, nơi mô hình lỗ kim không còn đúng
Depth Anything	tương đối (có bản có đơn vị)	Mở rộng dữ liệu bằng nhãn giả dưới nhiễu loạn mạnh; v2 dùng thầy huấn luyện trên ảnh tổng hợp	Cảnh xa mọi thứ trong tập ảnh không nhãn; mô hình lớn, chi phí cao
Marigold	không có	Tái dụng mô hình khuếch tán sinh ảnh làm bộ dự đoán độ sâu	Cần nhiều bước khử nhiễu; đầu ra <i>ngẫu nhiên</i> giữa các lần chạy

C. Bất biến tỉ lệ và dịch: hai ẩn tự do, và cái bẫy nằm ở ẩn thứ hai

Đây là chỗ phải viết đủ chứ không được rút gọn, vì rút gọn là chỗ mọi người vấp. Một mạng thuộc họ MiDaS trả về một đại lượng $\hat{d}$ liên hệ với độ sâu thật theo

$$\hat{d} \approx \frac{s}{Z} + t,$$

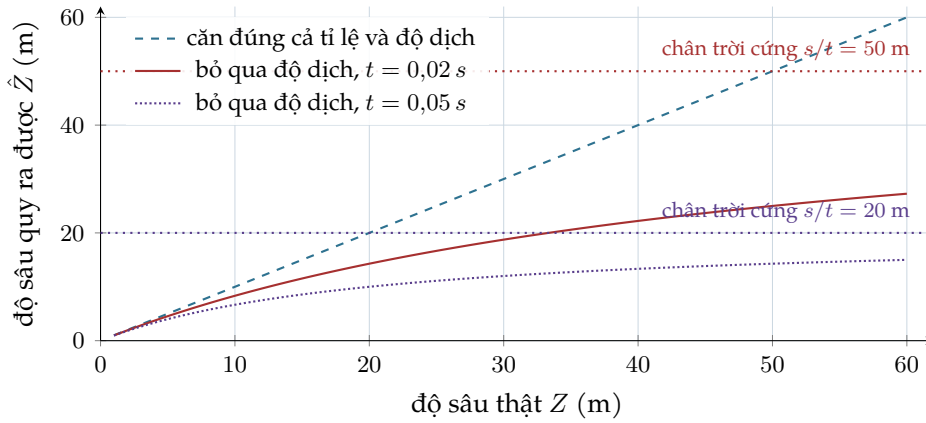
tức nó bất biến với một phép biến đổi bậc nhất trên *độ sâu nghịch đảo*. Nói bằng lời: mạng biết chắc điểm nào gần hơn điểm nào và gần hơn bao nhiêu lần, nhưng nó không biết gốc và không biết đơn vị.

Cách dùng ngây thơ nhất là chia một hằng số: lấy trung vị của $\hat{d}$ rồi co giãn cho khớp với trung vị độ sâu bạn đã biết. Làm thế là ngầm đặt $t = 0$. Hậu quả tính được ngay:

$$\hat{Z} = \frac{s}{\hat{d}} = \frac{s}{s/Z + t} = \frac{Z}{1 + tZ/s}.$$

Khi $Z \rightarrow \infty$, giá trị này không tiến ra vô cực mà bão hoà ở s/t . Mô hình của bạn vừa mọc ra một *chân trời cứng*: mọi thứ xa hơn s/t mét đều được báo về đúng khoảng đó.

Ví dụ nhỏ tính tay được. Giả sử sau khi căn tỉ lệ, độ dịch còn sót lại tương đương $t = 0,02 s$ trên thang độ sâu nghịch đảo, tức chân trời $s/t = 50$ m. Điểm ở 10 m được báo về $10/(1 + 0,2) = 8,3$ m, sai 17%. Điểm ở 20 m được báo về $20/(1,4) = 14,3$ m, sai 29%. Điểm ở 40 m được báo về 22 m, sai 45%. Sai số không ngẫu nhiên mà tăng đơn điệu theo khoảng cách — đúng dạng sai lệch có hệ thống, tức dạng nguy hiểm nhất cho một bộ tối ưu. Hình 1.3 vẽ chính hiện tượng đó.



Hình 1.3. Cái giá của việc bỏ quên ẩn thứ hai. Đầu ra của một mạng bất biến tỉ lệ và dịch cần *hai* tham số để quy về mét; cần bằng một hằng số nhân duy nhất tương đương đặt độ dịch bằng không, và điều đó tạo ra một chân trời cứng ở s/t mét. Đường liền đỏ và đường chấm tím là hai mức độ dịch còn sót khác nhau. Đây là hình minh hoạ công thức $\hat{Z} = Z/(1 + tZ/s)$, không phải số đo từ một mô hình cụ thể. Điểm cần nhớ: sai số tăng đơn điệu theo khoảng cách, nên nó là sai lệch có hệ thống chứ không phải nhiễu.

CẠM BẦY

Hai lỗi hay đi cùng nhau khi cắm một mô hình họ MiDaS vào pipeline. Thứ nhất là quên rằng đầu ra nằm trên thang *độ sâu nghịch đảo*, nên đem trung vị độ sâu ra co giãn thẳng là căn sai không gian, và sai số sinh ra sẽ tập trung hết vào vùng xa. Thứ hai là bỏ quên ẩn độ dịch, tạo ra chân trời cứng vừa nói ở trên. Nhưng lỗi thứ ba mới là lỗi giết hệ thống: s và t là của riêng từng khung hình. Bạn căn đúng cho khung k không có nghĩa là khung $k + 1$ dùng lại được cặp số đó. Một bức tường đứng yên sẽ “thở” phập phồng qua các khung, và bộ tối ưu của bạn — vốn tin rằng cảnh cứng — sẽ hấp thụ nhịp thở đó vào tư thế và vào vận tốc.

D. Metric3D: xoá một tính không xác định được bằng cách chuẩn hoá đầu vào

Mục này đáng đọc kỹ ngay cả khi bạn không định dùng Metric3D, vì thiết kế của nó là một bài học dùng lại được.

Bắt đầu từ nguồn gốc của tính không xác định được, lần này ở dạng khác với tính không xác định được của tỉ lệ. Với một camera lỗ kim, tọa độ ảnh của một điểm là $u = fX/Z$. Nhân đồng thời f và Z với cùng hệ số k thì u không đổi. Nghĩa là: cùng một bức ảnh có thể là một phòng nhỏ chụp bằng ống kính góc rộng, hoặc một nhà kho lớn chụp bằng ống kính tele. Một mạng chỉ nhìn pixel không có cách nào phân biệt hai trường hợp đó. Đây không phải khiếm khuyết của mạng; đó là thiếu thông tin thật.

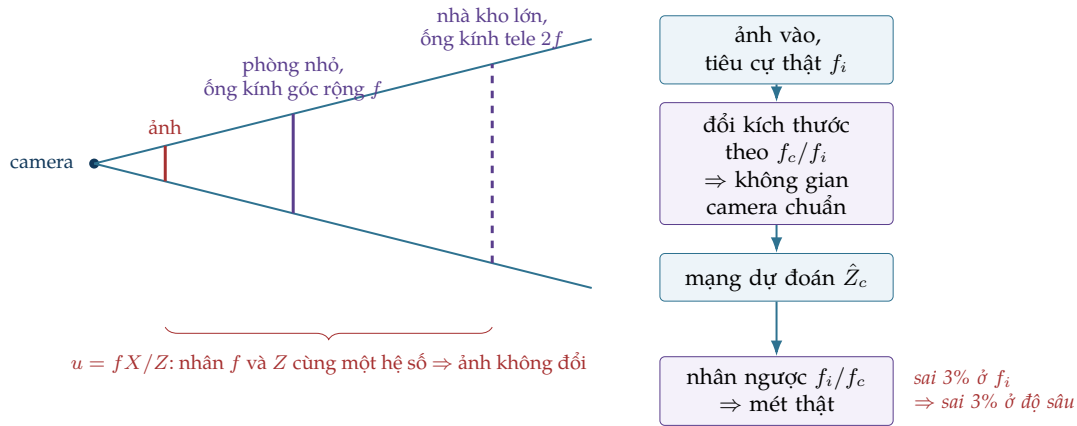
Điều đó giải thích một hiện tượng vốn khó hiểu khi huấn luyện: gộp nhiều tập dữ liệu chụp bằng nhiều camera khác nhau, có nhãn mét đầy đủ, mà mô hình vẫn học ra một thứ nhoè nhoẹt. Lý do là cùng một hình ảnh trong hai tập lại đi kèm hai nhãn mét khác nhau, chỉ vì hai camera có tiêu cự khác nhau. Mạng bị ép học một hàm không phải hàm.

Lời giải của Metric3D là gỡ ẩn số đó ra khỏi bài toán học, đưa nó về bước tiền xử lý. Chọn một tiêu cự chuẩn f_c . Với mỗi ảnh huấn luyện có tiêu cự thật f_i , hoặc thay đổi kích thước ảnh theo tỉ số f_c/f_i để tiêu cự hiệu dụng thành f_c , hoặc giữ ảnh và nhân nhãn độ sâu với f_c/f_i . Sau bước này, toàn bộ dữ liệu huấn luyện hành xử như thể chụp bằng một camera duy nhất, và ánh xạ từ ảnh sang mét trở lại là một hàm. Lúc chạy thật, ta đưa ảnh vào không gian chuẩn, dự đoán, rồi nhân ngược lại theo tiêu cự thật của camera mình.

Ý nghĩa bằng lời: đây chính là cố định gauge. Bạn đã làm đúng việc này mỗi lần cố định khung khoá đầu tiên trong một bài toán hiệu chỉnh chùm tia. Có một bậc tự do mà dữ liệu không nói gì về nó. Thay vì hy vọng bộ tối ưu tự xử lý, ta ghim nó lại bằng tay.

→ cùng nguyên lý với F/34 — đặt trọng số IMU cũng là bài toán “một đại lượng dữ liệu không nói gì tới”, và cách chữa cũng là ghim nó bằng một quy ước rõ ràng thay vì để bộ tối ưu đoán.

Cái giá thì rất thẳng thắn và rất dễ kiểm tra: đầu ra tuyến tính theo tiêu cự bạn khai báo. Nếu f_x hiệu chỉnh của bạn lệch 3%, độ sâu lệch đúng 3%, đều khắp, không có triệu chứng nào khác. Và trên ống kính mắt cá của một số cấu hình thực địa, “tiêu cự lỗ kim” thậm chí không phải một đại lượng xác định. Hình 1.4 vẽ lại toàn bộ lập luận.



Hình 1.4. Trái: vì sao độ sâu mét từ một ảnh là bài toán không giải được nếu không biết tiêu cự — hai cảnh khác nhau chụp bằng hai ống kính khác nhau cho đúng một ảnh. Phải: cách Metric3D gỡ nút. Ấn số bị đẩy ra khỏi bước học và bị ghim bằng một phép chuẩn hoá ở hai đầu, đúng kiểu cố định gauge. Hệ quả trực tiếp là độ chính xác của độ sâu bị chặn trên bởi độ chính xác của phép hiệu chỉnh tiêu cự.

E. Với một hệ stereo-inertial thì độ sâu học được thêm được gì

Bây giờ mới tới câu hỏi thật, và cách trả lời nó là bằng số chứ không bằng ý kiến.

Stereo cho độ sâu qua $Z = fb/d$, nên độ bất định của nó là $\Delta Z = Z^2 \Delta d / (fb)$. Đó là quan hệ bậc hai: sai số tăng theo bình phương khoảng cách. Lấy đúng con số của EuRoC làm ví dụ, với $f \approx 458$ pixel và $b \approx 0,11$ m, ta có $fb \approx 50,5$ đơn vị mét-pixel. Giả sử bộ ghép cặp của bạn đạt độ chính xác độ chênh lệch cỡ 0,2 pixel.

- Ở $Z = 5$ m: $\Delta Z = 25 \times 0,2/50,5 \approx 0,10$ m, tức 2%.
- Ở $Z = 15$ m: $\Delta Z = 225 \times 0,2/50,5 \approx 0,89$ m, tức 6%.
- Ở $Z = 30$ m: $\Delta Z = 900 \times 0,2/50,5 \approx 3,6$ m, tức 12%.

Ở khoảng 50 m, độ chênh lệch tụt xuống dưới một pixel và stereo của bạn về bản chất đã thành đơn mắt.

Điểm cần thấy không phải một con số mà là *hai hình dạng đường cong khác nhau*. Sai số stereo tăng theo Z^2 ; sai số tương đối của một mô hình độ sâu đơn ảnh, chạy trong miền nó đã học, về cơ bản *không* tăng theo khoảng cách, vì nó không dựa vào thị sai chút nào. Một đường cong tăng bậc hai và một đường cong gần như phẳng thì bắt buộc cắt nhau ở đâu đó. Xa hơn điểm cắt ấy, phỏng đoán học được đã tốt hơn phép đo của bạn; gần hơn, phép đo thắng.

Chương này cố ý *không* nói điểm cắt đó nằm ở bao nhiêu mét, vì một con số như thế đòi hai thứ mà không tài liệu nào cho bạn được: độ chính xác độ chênh lệch thật của bộ ghép cặp của bạn, và sai số thật của mô hình trên đúng loại cảnh của bạn. Cả hai đều đo được, và đo chúng chính là

nội dung mini project của chương. Nếu điểm cắt rơi ra ngoài dải khoảng cách mà cảnh của bạn có, thì cả hướng này đóng lại và bạn tiết kiệm được nhiều tuần.

Ba chỗ còn lại mà phép đo của bạn thật sự cạn:

- **Vùng ít vân** — tường trắng, hành lang, mặt sàn nhẵn. Không có vân thì không có ghép cặp, nên không có độ chênh lệch, nên không có độ sâu ở bất kỳ khoảng cách nào.
- **Khởi tạo khi thị sai chưa đủ** — lúc đứng yên hoặc lúc chỉ xoay tại chỗ. Với hệ stereo thì đường cơ sở cứu bạn, nên chỗ này ít quan trọng hơn nhiều so với hệ đơn mắt. Đây là lý do phải nói thẳng: rất nhiều lợi ích được quảng cáo trong các bài báo độ sâu học được là lợi ích của hệ đơn mắt, và chúng biến mất khi bạn có stereo.
- **Lắp lỗ cho bản đồ dày** — nếu bạn cần một bản đồ chiếm dụng để điều hướng chứ không phải để định vị. Đây là cách dùng an toàn nhất, vì nó nằm *hạ nguồn* của đồ thị tư thế và không phản hồi ngược vào nó.

F. Ba cách lấy tỉ lệ mét, và cách nào hợp với bạn

Nếu quyết định dùng một mô hình tương đối, bạn phải chọn một trong ba cách đưa nó về mét. Chúng không tương đương, và cách thứ ba là cách duy nhất bạn nên cân nhắc.

- **Căn theo chính phép đo stereo của bạn.** Ở mỗi khung khoá, lấy các điểm bản đồ đã có độ sâu đo được, khớp một phép biến đổi bậc nhất giữa dự đoán và phép đo trên tập điểm đó, rồi áp phép biến đổi ấy cho toàn ảnh. Ưu điểm: dùng đúng dữ liệu bạn tin. Nhược điểm nặng: cặp tham số được ước lượng từ vùng *gần*, nơi bạn có phép đo, rồi được ngoại suy sang vùng *xa*, nơi bạn cần nó. Ngoại suy một phép biến đổi bậc nhất ra ngoài miền khớp là đúng công thức để sai có hệ thống.
- **Căn theo IMU.** Dùng tích phân trước của gia tốc kế để cố định tỉ lệ trên một đoạn quỹ đạo, rồi buộc độ sâu theo đó. Vấn đề: gia tốc kế cố định tỉ lệ của *chuyển động*, không phải của cấu trúc, và nó chỉ làm được điều đó khi có gia tốc đủ kích thích. Trong một đoạn đi thẳng đều, nó không nói gì cả.
- **Dùng mô hình vốn đã ra mét.** Metric3D và các đầu ra có đơn vị của ZoeDepth hoặc Depth Anything không cần bước căn nào, nên không có chỗ nào để ngoại suy sai. Đổi lại bạn nhận toàn bộ rủi ro lệch miền và rủi ro sai tiêu cự.

Nhận xét thực dụng: hai cách đầu *tạo ra* một tham số mới ước lượng ở mỗi khung, tức tạo ra đúng cái nhíp thử mà mục sau nói là nguy hiểm. Cách thứ ba không tạo thêm tham số nào, nên nếu bạn buộc phải dùng độ sâu học được trong bộ tối ưu, hãy dùng mô hình ra mét và kiểm tra kỹ tiêu cự. Nếu chỉ dùng ngoài bộ tối ưu, cách nào cũng được.

G. Đo độ sâu học được cho đúng mục đích của bạn

Bảng xếp hạng của các *benchmark* độ sâu dùng một bộ thước quen thuộc: sai số tương đối tuyệt đối trung bình trên mọi pixel, và tỉ lệ pixel có sai số dưới một ngưỡng. Cả hai đều là trung bình trên toàn ảnh, và cả hai đều *giấu đi* đúng hai thứ quyết định với bạn.

Thứ nhất, chúng trộn lẫn mọi khoảng cách. Nhưng bạn không cần độ sâu ở mọi khoảng cách; bạn chỉ cần nó ở dải mà stereo đã cạn. Một mô hình đứng đầu bảng nhờ chính xác ở ba mét đầu có thể hoàn toàn vô dụng ở ba mươi mét. Phép đo đúng là vẽ sai số tương đối *theo khoảng cách*, rồi đọc riêng phần đuôi xa.

Thứ hai, và nặng hơn nhiều, chúng không nói gì về *cấu trúc* của sai số. Sai số trung bình 6% có thể là 6% nhiễu độc lập từng pixel, hoặc là 6% lệch chung cho cả mảng tường. Với bộ tối ưu của bạn, hai trường hợp đó cách nhau ba bậc độ lớn về mức nguy hiểm, và mục sau sẽ tính ra con số ấy. Không bảng xếp hạng nào phân biệt chúng.

Cách đo đúng thì đơn giản và bạn tự làm được. Với mỗi khung hình có chân trị độ sâu, tách sai số làm hai phần:

- **Phần chung của khung** — một cặp tỉ lệ và độ dịch tối ưu cho riêng khung đó, tìm bằng bình phương tối thiểu. Đây là phần mà một biến phụ trong bộ tối ưu có thể hấp thụ, nên nó *không* nguy hiểm nếu bạn mô hình hoá nó.
- **Phần dư sau khi trừ đi phần chung** — đây mới là thông tin thật mà mô hình đóng góp, và nó là thứ duy nhất đáng đem so giữa hai mô hình.

Rồi làm thêm một việc nữa: vẽ *phần chung* theo thời gian trên cả chuỗi. Nếu nó dao động ở tần số khung hình thì bạn đã nhìn thấy trước, bằng biểu đồ, đúng cái nhịp thở sẽ làm hỏng ước lượng vận tốc của mình — và bạn thấy nó trước khi viết một dòng mã tích hợp nào.

→ **cùng nguyên lý với E/24** — đây là phân rã lỗi theo loại, áp cho một mô-đun mới: tách phần sửa được bằng một tham số khỏi phần thật sự là sai số.

1.3 Dòng quang dày: từ FlowNet tới SEA-RAFT (Dense Optical Flow)

A. Vì sao ghép cặp thưa chết mà dòng quang sống

Chương 15 đã dựng bài toán khâu độ và Lucas–Kanade, nên ở đây ta đi thẳng vào chỗ liên quan tới nỗi đau của bạn: hai khung hình liên tiếp trong một pha xoay nhanh, ảnh bị mờ, và bộ bám đặc trưng mất dấu.

Cơ chế của cái chết đó đáng viết ra rõ ràng, vì nó giải thích luôn vì sao dòng quang lại sống sót. **Ảnh mờ do chuyển động** (*motion blur*) là một phép tích chập với một nhân dạng đoạn thẳng, dài

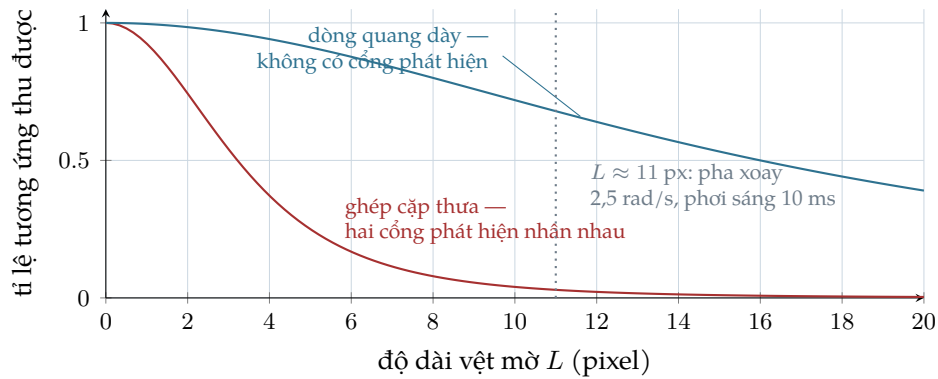
$$L \approx f \omega t_{\text{phơi sáng}} \quad \text{pixel},$$

với ω là tốc độ góc. Lấy số của EuRoC: $f \approx 458$ pixel, một pha xoay nhanh cỡ $\omega = 2,5$ rad/s, thời gian phơi sáng 10 ms trong nhà thiếu sáng. Ra $L \approx 11$ pixel. Đó là một vệt dài hơn cả bán kính của một mảnh mô tả ORB.

Bây giờ tới phần quan trọng. Bộ phát hiện góc đo hai giá trị riêng của ma trận mô men bậc hai của gradient trong một cửa sổ. Làm mờ theo một hướng là lọc thông thấp theo hướng đó: đỉnh gradient tụt đi khoảng L lần và trải ra trên L pixel, nên tổng bình phương gradient theo hướng ấy giảm cỡ L lần. Với $L \approx 11$, giá trị riêng nhỏ nhất mất khoảng một bậc độ lớn. Ngưỡng phát hiện là ngưỡng cứng, nên điểm đơn giản là *không được phát hiện nữa*. Đây là lập luận bậc độ lớn từ cơ chế, không phải số đo.

Và ghép cặp thưa là một chuỗi cổng nhân với nhau. Điểm phải được phát hiện ở khung này *và* ở khung kia, mô tả phải nằm trong ngưỡng, phải qua kiểm tra tỉ số, rồi phải sống sót qua RANSAC. Nếu tính lặp lại của bộ phát hiện tụt từ 0,8 xuống 0,3 thì xác suất một điểm sống qua cả hai khung tụt từ 0,64 xuống 0,09 — giảm bảy lần chỉ vì một cổng. Đó là lý do số điểm khớp sụp đổ chứ không giảm từ từ.

Dòng quang dày không có cổng nào cả. Nó không phát hiện gì hết; nó hỏi “pixel này đi đâu” cho mọi pixel, và nó trả lời bằng cách tổng hợp bằng chứng trên một vùng lớn. Làm mờ phá tần số cao nhưng giữ nguyên cấu trúc tần số thấp, mà tương quan ở mức thô lại đúng là thứ đọc cấu trúc tần số thấp. Thêm nữa, tính trơn học được của mạng lan nghiệm từ vùng có vân sang vùng mờ bên cạnh. Hình 1.5 vẽ hai đường cong tương ứng với hai cơ chế đó.



Hình 1.5. Vì sao ghép cặp thừa sụp đổ còn dòng quang chỉ xuống dốc. Hai đường này là *hình dạng của cơ chế*, không phải số đo: đường đỏ là tích của hai xác suất phát hiện nên nó bình phương mọi suy giảm, còn đường xanh không có cổng phát hiện nào để nhận. Kết luận dùng được: khi vật mờ vượt cỡ bán kính mảnh mô tả, khoảng cách giữa hai đường mở ra nhanh, và đó chính là dài mà một bộ dòng quang mới có lại.

B. Bốn thể hệ dòng quang học được

I Mạch 1 — dòng quang học được, từ FlowNet tới SEA-RAFT

- 1 Vấn đề** Phương pháp biến phân cổ điển chính xác nhưng chậm, và mỗi cảnh phải chỉnh lại tham số bằng tay.

Giải pháp FlowNet đưa cả bài toán vào một mạng tích chập huấn luyện đầu-cuối [10]. Có hai biến thể đáng nhớ vì chúng thể hiện hai triết lý: bản đơn giản xếp chồng hai ảnh thành sáu kênh rồi để mạng tự xoay xở, còn bản có tương quan cài sẵn một tầng tương quan giữa hai luồng đặc trưng.

Giả định cảnh trong dữ liệu tổng hợp dùng để huấn luyện đủ giống cảnh thật.

Vấn đề mới bản có tương quan thẳng rõ, cho thấy để mạng “tự học hình học” là lãng phí; nhưng độ chính xác vẫn thua các phương pháp cổ điển tốt.
- 2 Vấn đề** Một mạng đơn không đủ chính xác, và chuyển động nhỏ bị bỏ sót.

Giải pháp FlowNet 2.0 xếp chồng nhiều FlowNet nối tiếp, mỗi tầng *làm cong* ảnh thứ hai theo nghiệm của tầng trước rồi chỉ ước lượng phần dư; thêm một nhánh nhỏ chuyên cho chuyển động nhỏ và một khối hợp nhất ở cuối [11].

Hệ quả bài học lớn nhất của công trình này không phải kiến trúc mà là *liệu trình dữ liệu*: huấn luyện trên tập đơn giản trước rồi mới sang tập phức tạp cho kết quả khác hẳn so với trộn lẫn.

Giá phải trả mô hình rất lớn, và cấu trúc xếp chồng khiến mỗi tầng chỉ tinh chỉnh quanh nghiệm của tầng trước.
- 3 Vấn đề** Mạng quá to, và ba nguyên lý cổ điển đang bị học lại một cách lãng phí.

Giải pháp PWC-Net viết thẳng ba nguyên lý đó thành ba khối kiến trúc [12]: kim tự tháp đặc trưng học được, làm cong đặc trưng của ảnh hai theo nghiệm của mức thô hơn, và khối chi phí cục bộ với bán kính tìm kiếm nhỏ ở mỗi mức.

Hệ quả nhỏ hơn FlowNet 2.0 hơn một bậc mà chính xác hơn. Đây là ví dụ sạch nhất trong cả chương về giá trị của thiên lệch quy nạp đúng.

Giá phải trả thừa hưởng nguyên vẹn chế độ hồng của chiến lược thô tới mịn: vật nhỏ bay nhanh biến mất ở mức thô, và khối chi phí cục bộ không đủ tầm để sửa lại.

- 4 Vấn đề** Sai ở mức thô là sai vĩnh viễn, vì mọi mức mịn hơn chỉ tinh chỉnh chứ không tìm lại.
- Giải pháp** RAFT tách hẳn hai việc [13]: tính *một lần* khối tương quan giữa mọi cặp vị trí, rồi lặp lại một phép tra cứu cục bộ quanh phỏng đoán hiện tại, với phép cập nhật do một mạng hồi tiếp đảm nhiệm.
- Giả định** khối tương quan toàn cặp vừa bộ nhớ.
- Hệ quả** chuyển động lớn không còn cần thu nhỏ ảnh, nên vật nhỏ bay nhanh không biến mất. Số vòng lặp trở thành một núm vắn độ trễ chỉnh được lúc chạy.
- Giá phải trả** bộ nhớ của khối tương quan, và một mô hình mà bạn không ablate được từng phần.
- 5 Vấn đề** Phép tra cứu của RAFT chỉ nhìn cục bộ, nên vùng bị che và vùng không vắn phải chờ thông tin lan qua nhiều vòng lặp.
- Giải pháp** FlowFormer mã hoá chính khối chi phí bằng một *transformer*, để thông tin đi khắp khối trong một lượt thay vì bò dần qua các vòng lặp [14].
- Giá phải trả** bộ nhớ và độ trễ tăng mạnh; đây là hướng cho máy để bàn, không cho hệ nhúng.
- 6 Vấn đề** RAFT cần nhiều vòng lặp vì nó khởi động từ dòng bằng không, và nó không nói được nó tin vào pixel nào.
- Giải pháp** SEA-RAFT đổi ba thứ [15]: hồi quy thẳng một nghiệm khởi đầu thay vì bắt đầu từ không, đổi hàm mất mát sang hỗn hợp hai phân phối Laplace, và đổi dữ liệu tiền huấn luyện.
- Hệ quả** ít vòng lặp hơn nhiều nên nhanh hơn hẳn; và hàm mất mát hỗn hợp cho ra một *độ bất định theo từng pixel* dùng được làm trọng số.
- Giả định** hai thành phần Laplace đủ để mô tả tính đa phương thức của dòng ở biên vật thể.

RAFT-3D là một nhánh đáng biết vì nó đổi đầu ra chứ không đổi thuật toán [16]. Thay vì một vector dịch chuyển hai chiều cho mỗi pixel, nó dự đoán một phép biến đổi cứng ba chiều cho mỗi pixel, rồi ép các pixel có biểu diễn giống nhau phải chia sẻ cùng một chuyển động cứng. Hệ quả trực tiếp và rất hữu ích: các mảng cứng của cảnh tự tách ra thành cụm. Nói cách khác, bài toán phân tách vật thể động rơi ra như một sản phẩm phụ của biểu diễn, không cần một bộ phân vùng ngữ nghĩa riêng.

↔ **đối lập với** F/33 — đó là hai cách khác nhau để loại vật động: RAFT-3D dùng tính không nhất quán của chuyển động, còn mặt nạ ngữ nghĩa dùng tên lớp của vật.

C. Vòng lặp tra cứu của RAFT, giải thích kỹ

Đây là thiết kế đáng học nhất trong cả mục, và nó đáng học ngay cả khi bạn không bao giờ chạy RAFT.

Ba thành phần, theo đúng thứ tự dữ liệu chảy qua:

- Mã hoá đặc trưng.** Cả hai ảnh được đưa qua cùng một bộ mã hoá xuống độ phân giải 1/8. Một bộ mã hoá thứ hai, chỉ chạy trên ảnh một, cho ra đặc trưng ngữ cảnh nuôi vào phép cập nhật.
- Khối tương quan toàn cặp.** Với mọi vị trí i trên lưới ảnh một và mọi vị trí j trên lưới ảnh hai, tính tích vô hướng của hai vector đặc trưng. Với ảnh EuRoC 752×480 , lưới 1/8 là $94 \times 60 = 5640$ vị trí, nên khối có 5640^2 phần tử — *bình phương số ô lưới*, và đó là con số đáng nhớ vì nó nói rằng độ phân giải lưới là núm vắn chi phí mạnh nhất. Rồi *gộp trung bình theo hai chiều cuối* với các hệ số 1, 2, 4, 8 để được bốn mức. Đây là câu quan trọng nhất của cả mục: cái bị làm thô là

hai chiều của ảnh hai trong khối tương quan, không phải lưới của ảnh một. Lưới ảnh một giữ nguyên độ phân giải ở mọi mức, nên một vật nhỏ ở ảnh một không bao giờ biến mất.

3. **Vòng lặp tra cứu và cập nhật.** Trường dòng khởi đầu bằng không. Ở mỗi vòng, với mỗi vị trí x của ảnh một, ta tính vị trí ứng viên $x' = x + f_k(x)$ rồi đọc các giá trị tương quan trong một lân cận bán kính 4 quanh x' , ở cả bốn mức. Mỗi vòng vì thế lấy ra $4 \times (2 \cdot 4 + 1)^2 = 324$ con số cho mỗi pixel. Ba thứ — các con số tra cứu được, dòng hiện tại, và đặc trưng ngữ cảnh — được đưa vào một khối hồi tiếp tích chập, khối này trả về Δf , và ta cộng vào. Lặp lại, dùng chung trọng số ở mọi vòng.

Ý nghĩa của thiết kế này bằng lời: nó tách bạch “chỗ nào giống chỗ nào” — tính một lần, đắt, nhưng không phụ thuộc nghiệm — khỏi “tôi đang đoán ở đâu” — rẻ, lặp lại, phụ thuộc nghiệm. Phép tra cứu ở mức thô thứ tư với bán kính 4 phủ được $\pm 4 \times 8 = \pm 32$ ô của lưới 1/8, tức ± 256 pixel trong ảnh gốc. Chuyển động lớn được xử lý mà không hề thu nhỏ ảnh một lần nào.

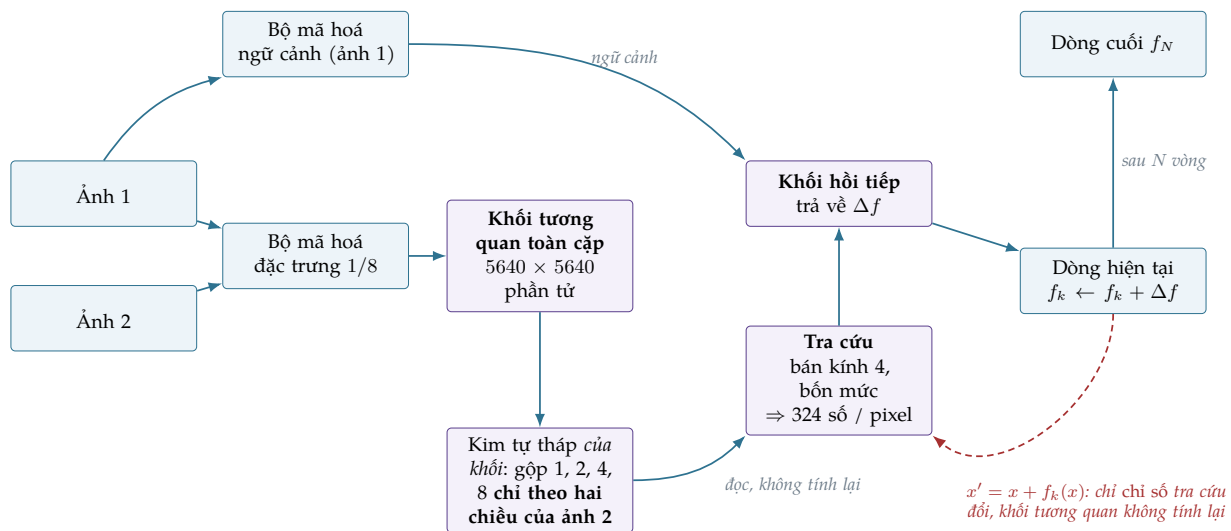
Bảng 1.2. Độ phủ của một phép tra cứu bán kính 4 ở bốn mức của kim tự tháp khối tương quan, quy về pixel của ảnh gốc. Cột cuối cho thấy vì sao không cần thu nhỏ ảnh: mức thô nhất đã phủ quá đủ cho mọi chuyển động thực tế giữa hai khung hình liên tiếp, trong khi lưới của ảnh một vẫn giữ nguyên độ phân giải ở cả bốn mức.

Mức	Hệ số gộp	Ô lưới 1/8	Pixel ảnh gốc	Vai trò
0	1	± 4	± 32	Tinh chỉnh dưới pixel ở các vòng cuối
1	2	± 8	± 64	Chuyển động vừa
2	4	± 16	± 128	Chuyển động nhanh
3	8	± 32	± 256	Xoay mạnh, mất khung, tìm lại từ xa

Với người đã viết bộ tối ưu phi tuyến, khối hồi tiếp kia chính là một *bước tối ưu học được*: nó nhận phần dư quanh nghiệm hiện tại và trả về một bước đi, đúng vai trò của một bước Gauss–Newton, chỉ khác là hướng đi được học chứ không được suy ra từ Jacobi. Và như mọi bộ giải lặp, số vòng lặp là ngân sách bạn tự đặt lúc chạy — huấn luyện với 12 vòng, chạy với 32 vòng nếu rảnh, hoặc 6 vòng nếu gấp.

↔ **trường hợp riêng của** C/15/03 — cùng một trực giác về bộ giải lặp có ngân sách, chỉ khác là ở đó số vòng lặp do bạn đặt còn ở đây nó là siêu tham số lúc suy luận.

Hình 1.6 là sơ đồ của đúng vòng lặp đó.



Hình 1.6. Vòng lặp tra cứu của RAFT. Phần bên trái chạy đúng một lần và không phụ thuộc nghiệm; phần bên phải chạy lặp và rẻ, chỉ đổi *chỉ số* tra cứu chứ không tính lại tương quan. Chi tiết quyết định nằm ở ô kim tự tháp: việc gộp trung bình chỉ áp lên hai chiều của ảnh hai, nên lưới của ảnh một giữ nguyên độ phân giải ở mọi mức. Đó là lý do RAFT xử lý được chuyển động lớn mà không mắc bẫy “vật nhỏ biến mất ở mức thô” của chiến lược thô tới mịn. Số vòng lặp là siêu tham số lúc suy luận, nên nó là núm vặn độ trễ chỉnh được ngay khi chạy.

E. Ba cách cắm dòng quang vào hệ của bạn, xếp theo mức xâm lấn

Trước khi tính giá, cần thấy rằng “dùng dòng quang” không phải một lựa chọn mà là ba lựa chọn rất khác nhau. Xếp theo mức xâm lấn tăng dần:

1. **Dùng làm bộ dự đoán, giữ nguyên ghép cặp thừa.** Chạy dòng quang, lấy vector dịch chuyển tại vị trí các điểm bản đồ đang bám, rồi dùng nó để đặt tâm cửa sổ tìm kiếm cho bộ ghép cặp thừa vốn có. Mọi bài kiểm tra cũ vẫn chạy: kiểm tra tỉ số, kiểm tra sai số chiếu lại, RANSAC. Bạn chỉ đổi *chỗ nhìn*, không đổi *cái tin*.
2. **Dùng làm nguồn tương ứng, bỏ bộ phát hiện.** Lấy thẳng trường dòng làm tập tương ứng, lấy mẫu thừa ra vài trăm điểm theo một tiêu chí nào đó, rồi đưa vào bộ giải. Bạn mất bài kiểm tra tỉ số vì không còn hai ứng viên để so, và bạn phải tự dựng lại một cơ chế loại điểm sai.
3. **Dùng làm phép đo dày, đưa thẳng vào bộ tối ưu.** Đây là con đường của các hệ đầu-cuối ở chương 31. Bạn được nhiều ràng buộc nhất và mất nhiều khả năng chẩn đoán nhất.

Với một hệ đã tinh chỉnh kỹ, lựa chọn thứ nhất gần như luôn là lựa chọn đúng để thử trước, vì lý do rất thực dụng: nó *có thể tắt được*. Nếu dòng quang cho dự đoán tồi, cửa sổ tìm kiếm đặt sai chỗ, số điểm khớp giảm, và bạn thấy ngay bằng đúng các bộ đếm đang có. Không có gì âm thầm cả. Lựa chọn thứ hai và thứ ba đều xoá mất tín hiệu “không còn điểm nội biên nào” — tín hiệu duy nhất mà logic phục hồi của bạn đang dựa vào.

Có một chi tiết dễ bỏ sót ở lựa chọn thứ nhất và nó quyết định lời lỗ. Hệ của bạn đã có một bộ dự đoán rồi: mô hình chuyển động hằng vận tốc, hoặc tốt hơn nữa là tích phân trước của IMU. Trong pha xoay nhanh, IMU chính là thứ dự đoán tốt nhất, vì con quay hồi chuyển đo trực tiếp cái gây ra dịch chuyển ảnh. Vậy dòng quang chỉ có lãi ở đúng những khung hình mà dự đoán từ IMU cũng hỏng: khi có tịnh tiến nhanh ở cự ly gần, khi mất đồng bộ thời gian, hoặc khi độ lệch con quay đang trôi. Đó là một tập khung hình hẹp hơn nhiều so với “mọi khung bị mờ”, và đo được nó chính là nội dung mini project thứ hai của chương.

↔ **đối lập với** F/34 — IMU và dòng quang là hai bộ dự đoán cạnh tranh cho cùng một cửa sổ tìm kiếm; chọn cái nào là câu hỏi về việc cái nào hỏng trước.

F. Cái giá, nói bằng số

Bốn khoản, xếp theo mức độ dễ bị bỏ quên.

Bộ nhớ và độ trễ. Điều phải nắm là *cấu trúc* của chi phí, vì nó khác hẳn cấu trúc chi phí của một mạng thông thường. Khối tương quan toàn cặp có số phần tử bằng bình phương số ô lưới, và nó phải nằm nguyên trong bộ nhớ suốt các vòng lặp tra cứu. Hai hệ quả đi kèm. Thứ nhất, chi phí bộ nhớ tăng theo *bình phương* độ phân giải lưới, nên hạ độ phân giải là núm vặn mạnh nhất bạn có. Thứ hai, sau khi khối đã dựng xong thì mỗi vòng lặp chỉ là đọc bộ nhớ, nên bài toán chuyển từ bị chặn bởi phép tính sang bị chặn bởi băng thông. Con số mili-giây và megabyte cụ thể thì phải đo trên chính phần cứng đích, sau khi đã biên dịch bằng trình suy luận của nó — và mini project thứ hai của chương bắt bạn đo đúng con số ấy thay vì ước lượng nó.

↪ **ràng buộc bởi** B/11 — bài toán ở đây bị chặn bởi bộ nhớ và băng thông chứ không phải bởi số phép tính, đúng như phân tích roofline ở chương giới hạn tính toán.

Không có tập điểm nội biên. Đây là khoản đắt nhất mà ít ai nói. Pipeline thừa của bạn trả về một tập điểm khớp *kèm nhãn đúng hay sai*, và khi mọi thứ hỏng thì nó trả về con số không — một tín hiệu rõ ràng để kích hoạt logic phục hồi. Dòng quang luôn trả lời. Ở vùng bị che, ở vùng ra khỏi khung, ở mặt gương, nó vẫn cho ra một vector trông rất thuyết phục. Hệ cổ điển hỏng âm ỉ; hệ học được hỏng lạng lẽ.

Nó chỉ thay bước ghép cặp. Dòng quang cho tương ứng hai chiều. Bạn vẫn cần độ sâu và vẫn cần một bộ giải để ra tư thế. Nó không rút ngắn pipeline, nó chỉ đổi một khối.

Chẩn đoán và tính lặp lại. Bạn vừa đổi một khối có thể ablate từng phần lấy một khối chỉ ablate được nguyên cục. Bù lại, đây đúng là chỗ SEA-RAFT có giá trị đặc biệt với bạn: hàm mất mát hỗn hợp Laplace cho ra một độ bất định theo từng pixel, và độ bất định đó chuyển thẳng thành trọng số trong bài toán bình phương tối thiểu. Bài học thiết kế rút ra được, dùng lại được ở chỗ khác: *muốn có trọng số dùng được thì phải huấn luyện cho nó*. Một hàm mất mát đơn phương thức như L_1 không có cách nào diễn đạt “pixel này nằm trên biên che khuất nên câu trả lời có hai đáp án”.

↪ **tiền đề** F/34 — đó chính là cầu nối sang bài toán hiệp phương sai học được: trọng số trong đồ thị nhân tử phải đến từ một mô hình có khả năng biểu diễn tính đa phương thức.

1.4 Mô hình nền hình học: DUST3R, MAST3R và VGGT (Geometric Foundation Models)

A. Ý tưởng: đối biểu diễn đầu ra để xoá cả một chuỗi bước

Đây là hướng mới nhất và đáng chú ý nhất trong cả chương, và lý do đáng chú ý không nằm ở độ chính xác mà ở chỗ nó *xoá bỏ* cái gì.

Pipeline hình học cổ điển là một chuỗi dài: phát hiện điểm, mô tả, ghép cặp, ước lượng ma trận cơ bản, phân rã ra tư thế, tam giác hoá, rồi hiệu chỉnh chùm tia. Mỗi bước có giả định riêng, và mỗi bước hỏng theo kiểu riêng. Ước lượng tư thế đặc biệt mong manh: nó cần đủ điểm khớp đúng, nó suy biến khi các điểm gần đồng phẳng hoặc khi chuyển động thuần xoay, và nó luôn cần biết tiêu cự.

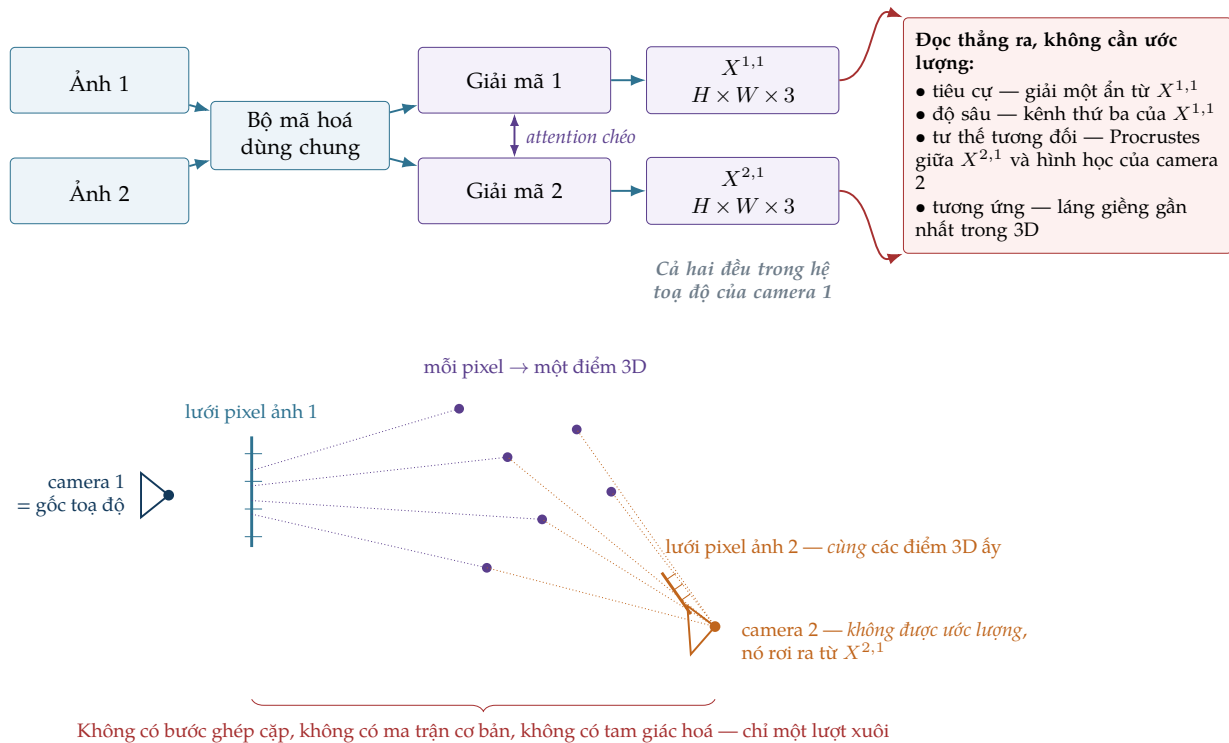
DUST3R [17] đặt câu hỏi ngược: nếu đầu ra của mạng không phải độ sâu mà là *bản đồ điểm* thì cần bao nhiêu bước trong chuỗi trên. Cụ thể, đưa vào hai ảnh chưa hiệu chỉnh, mạng trả về hai tensor $H \times W \times 3$: mỗi pixel của ảnh một được gán một điểm ba chiều, và mỗi pixel của ảnh hai cũng được gán một điểm ba chiều — nhưng *cả hai đều nằm trong hệ toạ độ của camera một*.

Câu “cả hai đều nằm trong hệ toạ độ của camera một” chính là toàn bộ mẹo. Tư thế tương đối không biến mất; nó bị *gói vào* đầu ra. Giả sử tôi biết mỗi pixel của ảnh hai ứng với điểm ba chiều nào trong hệ của camera một. Khi đó tư thế của camera hai chỉ là phép biến đổi cứng khớp bản

đồ điểm ấy với hình học của chính nó. Đó là một bài toán Procrustes hoặc PnP, có nghiệm dạng đóng, không cần RANSAC trên ma trận cơ bản. Tương tự, tiêu cự đọc ra được bằng cách giải một ẩn từ bản đồ điểm của ảnh một; độ sâu là kênh thứ ba; và tương ứng giữa hai ảnh là láng giềng gần nhất trong không gian ba chiều.

Kiến trúc thì đơn giản đến mức đáng ngạc nhiên: một bộ mã hoá *transformer* dùng chung cho hai ảnh, rồi hai bộ giải mã *liên tục nhìn sang nhau* bằng *attention chéo*, rồi hai đầu ra. Hàm mất mát là hồi quy trong không gian ba chiều, sau khi chuẩn hoá cả dự đoán lẫn nhãn về cùng một thang, và có trọng số theo một độ tin cậy do chính mạng dự đoán. Chuẩn hoá về cùng thang là chỗ tỉ lệ tuyệt đối bị bỏ đi, nên đầu ra gốc của DUST3R không có đơn vị mét.

Hình 1.7 vẽ cả cơ chế lẫn ý nghĩa hình học.



Hình 1.7. Bản đồ điểm của DUST3R: cơ chế ở trên, ý nghĩa hình học ở dưới. Mạng gán thẳng cho mỗi pixel của *cả hai* ảnh một điểm ba chiều trong *cùng* hệ toạ độ của camera một. Vì đã ở chung một hệ, mọi đại lượng mà pipeline cổ điển phải ước lượng qua nhiều bước đều trở thành phép đọc: tiêu cự, độ sâu, tư thế tương đối và tương ứng. Bài học thiết kế: chọn đúng biểu diễn đầu ra xoá được nhiều bước hơn là chọn đúng kiến trúc. Cái không xoá được là tỉ lệ tuyệt đối — hàm mất mát chuẩn hoá thang trước khi so, nên đầu ra không có đơn vị mét.

B. MAST3R, MAST3R-SLAM, VGGT, VGGT-SLAM

I MẠCH 2 — từ hai ảnh tới một hệ SLAM

1 Vấn đề

DUST3R cho hình học tốt nhưng tương ứng của nó chỉ là láng giềng gần nhất trong không gian ba chiều, nên không đủ chính xác để làm bộ ghép cặp.

Giải pháp

MASt3R gắn thêm một đầu ra mô tả đặc trưng dày lên chính kiến trúc đó, huấn luyện bằng một hàm mất mát ghép cặp, cộng một thủ tục ghép tương hỗ nhanh để tránh so sánh mọi cặp pixel [18].

Hệ quả

nó trở thành một bộ ghép cặp *được neo trong ba chiều*, nên nó chịu được thay đổi góc nhìn cực đoan — đúng trường hợp mà mọi bộ mô tả hai chiều đều chết.

- Giá phải trả** vẫn là một mô hình lớn chạy trên *từng cặp* ảnh, tức chậm hơn một bộ ghép cặp thừa nhiều bậc, và chi phí nhân lên theo số cặp.
- 2 Vấn đề** Có một bộ ghép cặp mạnh vẫn chưa phải một hệ SLAM: thiếu quản lý khung khoá, thiếu tối ưu toàn cục, thiếu đóng vòng.
- Giải pháp** MAST3R-SLAM dựng một hệ SLAM dày, chạy trực tuyến, đặt thẳng trên tiên nghiệm hai ảnh đó, và chạy được cả khi camera chưa hiệu chỉnh [19].
- Hệ quả** điểm thiết kế đáng học: nó tối ưu *trong không gian bản đồ điểm* chứ không trong không gian sai số chiếu lại, vì bản đồ điểm mới là thứ mạng cho ra.
- Giả định** phần cứng có GPU rời đủ mạnh. Đây là công trình rất mới; tôi nêu cơ chế mà không nêu con số hiệu năng.
- 3 Vấn đề** Suy luận theo từng cặp có chi phí bậc hai theo số ảnh, và phải có một bước căn chỉnh toàn cục riêng để ghép các cặp lại.
- Giải pháp** VGGT bỏ hẳn cấu trúc theo cặp: một *transformer* lớn nhận cả nhóm ảnh cùng lúc và trả về trong một lượt xuôi cả nội tham số, ngoại tham số, bản đồ độ sâu, bản đồ điểm và vết điểm ba chiều [20].
- Giải pháp** cơ chế cốt lõi là *xen kẽ hai loại attention*: một lượt attention trong phạm vi từng ảnh, rồi một lượt attention trên toàn bộ token của mọi ảnh, lặp lại nhiều lần. Lượt trong ảnh giữ chi tiết, lượt toàn cục làm cho các ảnh biết về nhau.
- Hệ quả** không còn bước căn chỉnh toàn cục nào cả; hình học ra thẳng, trong hệ toạ độ của camera đầu tiên.
- Giá phải trả** attention toàn cục trên mọi token của mọi khung là chỗ bộ nhớ nổ, và nó đặt một trần cứng lên số khung hình — mục sau tính con số đó.
- 4 Vấn đề** Nếu ghép các mảnh bản đồ do VGGT sinh ra bằng một phép biến đổi tương tự thì kết quả sai, chứ không chỉ là chưa tối ưu.
- Giải pháp** VGGT-SLAM nhận ra rằng với camera chưa hiệu chỉnh và cảnh gần phẳng, hai mảnh bản đồ liên hệ với nhau bằng một phép biến đổi xạ ảnh tổng quát chứ không phải phép tương tự; nên nó tối ưu trên nhóm $SL(4)$ mười lăm bậc tự do [21].
- Hệ quả** đây là bài học thiết kế đắt giá nhất của cả mục: *lớp bất định của đầu ra frontend quyết định đa tạp mà backend phải tối ưu trên đó*. Chọn nhầm đa tạp là sai mô hình, không phải chậm hội tụ.
- Giả định** cảnh và cấu hình camera thật sự rơi vào lớp bất định xạ ảnh đó. Công trình rất mới, tôi nêu ý tưởng chứ không nêu kết quả.

C. Hai kiểu trần số khung, và vì sao cả hai đều là trần

Đây là phần quyết định các phương pháp này cắm vào đâu được trong một hệ nhúng. Điểm mấu chốt không phải một con số megabyte — con số ấy đổi theo bản cài đặt và theo phần cứng, nên nêu nó ra ở đây là gây hiểu nhầm — mà là *dạng* của quan hệ giữa chi phí và số khung hình. Hai nhánh có hai dạng khác nhau, và chúng dẫn tới hai kiểu trần khác nhau.

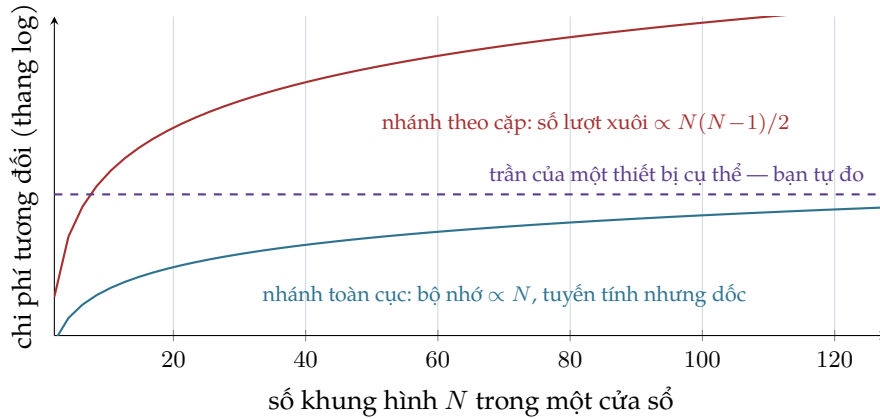
Nhánh theo cặp (DUST3R, MAST3R). Với N ảnh, đồ thị liên kết đầy đủ có $N(N - 1)/2$ cặp, và *mỗi cặp là một lượt xuôi riêng*. Đây là quan hệ bậc hai theo số khung, và nó là thứ chặn bạn chứ không phải bộ nhớ: một cửa sổ vài chục khung đã kéo theo hàng trăm lượt xuôi cho một lần cập nhật. Bộ nhớ thì tăng tuyến tính và hiền hơn nhiều, vì mỗi khung chỉ giữ một bản đồ điểm cỡ ba kênh trên mỗi pixel.

Nhánh toàn cục (VGGT). Ở đây chi phí đi theo số *token*, và số token là đại lượng đáng đếm vì nó suy ra được từ kiến trúc: một ảnh 518×518 chia thành mảnh 14×14 cho $37 \times 37 = 1369$ token

mỗi khung, nên N khung cho $1369N$ token. Attention *toàn cục* chạy trên toàn bộ chỗ token ấy, và mọi token phải giữ khoá cùng giá trị ở mọi tầng của thân *transformer*. Quan hệ vì thế tuyến tính theo N — nhưng với một hệ số nhân rất lớn, vì hệ số ấy là số tầng nhân chiều ẩn. Tuyến tính với hệ số lớn vẫn chạm trần rất sớm.

Cách đọc đúng hai đoạn trên: đừng nhớ megabyte, hãy nhớ *số mũ*. Nhánh theo cặp là bậc hai theo số khung; nhánh toàn cục là bậc một nhưng dốc. Muốn biết trần cụ thể của mình nằm ở đâu thì chạy một lượt xuôi với số khung tăng dần trên đúng phần cứng đích rồi đọc bộ nhớ đỉnh — một buổi chiều, và kết quả đúng cho máy của bạn thay vì đúng cho máy của người viết bài báo.

Hình 1.8 vẽ lại hai dạng ấy. Kết luận kiến trúc rút ra được ngay và không phụ thuộc con số: cả hai nhánh đều có một trần cứng theo số khung, nên mọi hệ SLAM dựng trên nhóm này đều buộc phải chia bản đồ thành các mảnh nhỏ rồi ghép lại — và bước ghép ấy chính là chỗ VGGT-SLAM phát hiện ra rằng phép biến đổi tương tự là sai mô hình.



Hình 1.8. Hai kiểu trần số khung, vẽ trên thang log. Trục đứng cố ý không có vạch số: hình này nói về *số mũ*, không về megabyte, vì con số tuyệt đối đổi theo bản cài đặt và theo phần cứng còn số mũ thì không. Nhánh theo cặp bị chặn bởi số lượt xuôi tăng bậc hai — qua vài chục khung thì số cặp đã lên hàng nghìn, và đó là thứ chặn bạn trước cả bộ nhớ. Nhánh attention toàn cục tăng tuyến tính, nhưng hệ số nhân là số tầng nhân chiều ẩn, nên nó vẫn chạm trần sớm. Đường ngang là trần của một thiết bị cụ thể; vị trí thật của nó là thứ bạn đo bằng cách tăng dần số khung trên đúng phần cứng đích rồi đọc bộ nhớ đỉnh. Hệ quả kiến trúc không phụ thuộc con số: mọi hệ SLAM dựng trên nhóm này đều buộc phải chia bản đồ thành mảnh nhỏ rồi ghép lại.

Bảng 1.3. Năm công trình trong nhánh mô hình nền hình học. Cột cuối là cột quyết định với một hệ nhúng thời gian thực, và nó ghi *loại* ràng buộc chứ không ghi con số: mọi nhận định ở đây lấy từ mô tả phương pháp, không từ số đo mà tôi tự chạy, nên chúng nói được cái gì chặn bạn nhưng không nói được chặn ở mức nào.

Công trình	Cơ chế cốt lõi	Thêm được gì	Hỏng khi nào / cái giá
DUST3R	Hồi quy bản đồ điểm cho cặp ảnh, cả hai trong hệ của camera một	Bỏ hẳn ghép cặp, ma trận cơ bản và tam giác hoá	Không có đơn vị mét; nhiều ảnh thì chi phí theo cặp tăng bậc hai
MASt3R	Thêm đầu ra mô tả dày và ghép tương hỗ nhanh lên DUST3R	Ghép cặp neo trong ba chiều, chịu được góc nhìn cực đoan	Một lượt xuôi mô hình lớn cho <i>mỗi cặp</i> ; chi phí nhân theo số cặp
MASt3R-SLAM	Hệ SLAM dày trực tuyến, tối ưu trong không gian bản đồ điểm	Chạy được cả khi camera chưa hiệu chỉnh	Cần GPU rời mạnh; rất mới, ít bằng chứng ngoài phòng thí nghiệm
VGGT	Một <i>transformer</i> nhận nhiều ảnh, xen kẽ attention trong ảnh và toàn cục	Bỏ luôn bước căn chỉnh toàn cục; ra thẳng tư thế, độ sâu, bản đồ điểm	Bộ nhớ attention toàn cục đặt trần cứng lên số khung
VGGT-SLAM	Ghép các mảnh bản đồ của VGGT trên nhóm xạ ảnh SL(4)	Chỉ ra rằng phép biến đổi tương tự là <i>sai mô hình</i> khi camera chưa hiệu chỉnh	Chỉ đúng khi lớp bất định thật sự là xạ ảnh; rất mới

D. Chỗ nhánh này chạm vào nỗi đau đóng vòng

Có một chỗ mà nhánh này đáng để bạn theo dõi nghiêm túc, và nó không phải chỗ được quảng cáo nhiều nhất.

Chương 29 tách đóng vòng thành bốn tầng và chỉ ra rằng tầng kiểm chứng hình học mới là chỗ hay giết vòng nhất, đặc biệt khi hai lần đi qua cùng một chỗ theo hai hướng ngược nhau. Đó chính xác là trường hợp mà một bộ mô tả hai chiều không cứu được: cùng một góc tường nhìn từ hai phía cho hai mảnh ảnh không có gì chung. MASt3R ghép cặp trong không gian ba chiều, nên nó không phụ thuộc vào việc hai mảnh ảnh trông giống nhau; nó phụ thuộc vào việc hai bản đồ điểm mô tả cùng một hình học.

→ **hệ quả của F/29** — đây là một lối thoát khá dễ cho đúng cái nút thắt mà chương 29 đo được ở tầng kiểm chứng hình học.

Cái giá thì rõ, và nó là lập luận về *tần suất* chứ không về mili-giây: đóng vòng chạy không thường xuyên và không nằm trên đường tới hạn thời gian thực, nên một mô hình đắt gấp nhiều lần ngân sách một khung hình vẫn dùng được ở đó, trong khi cùng mô hình ấy đặt vào *frontend* thì không. Nói cách khác, nếu có một chỗ trong hệ của bạn mà nhánh mô hình nền hình học đáng thử trước tiên, thì đó là tầng kiểm chứng ứng viên vòng, không phải *frontend*.

E. Tối ưu trong không gian bản đồ điểm: được gì và mất gì

MASt3R-SLAM tối ưu trong không gian bản đồ điểm chứ không trong không gian sai số chiều lại. Chi tiết ấy nghe như một lựa chọn cài đặt, nhưng nó đổi bản chất bài toán, và người đã viết bộ hiệu chỉnh chùm tia sẽ thấy ngay vì sao.

Sai số chiều lại có một tính chất mà ta hay quên vì nó quá quen: nó là sai số *trong không gian đo*. Nhiều của bộ phát hiện điểm gần với nhiều đẳng hướng vài phần mười pixel, nên bình phương tối thiểu trên sai số chiều lại xấp xỉ đúng ước lượng hợp lý cực đại. Chuyển sang đo sai lệch trong không gian ba chiều phá vỡ điều đó. Một sai lệch nửa pixel ở một điểm cách ba mét tương ứng

vài milimet trong không gian; cùng nửa pixel ấy ở ba mươi mét tương ứng vài chục centimet. Đo bằng mét là ngầm gán cho điểm xa một trọng số lớn hơn hàng trăm lần so với điều mà phép đo thật sự đáng được hưởng.

Vậy tại sao vẫn làm? Vì mạng không cho bạn phép đo trong không gian ảnh; nó cho bạn bản đồ điểm. Ép nó quay về không gian ảnh đòi hỏi biết nội tham số và đòi hỏi tin vào đó, mà một trong những điểm bán hàng của cả nhánh này lại là không cần hiệu chỉnh. Đây là một đánh đổi thật, không phải một sơ suất — nhưng nó giải thích một hiện tượng bạn sẽ thấy khi đọc kết quả của nhánh này: chất lượng dựng lại nhìn rất đẹp trong khi độ chính xác quỹ đạo thì chưa bằng một hệ cổ điển đã tinh chỉnh kỹ.

F. Lỗ hổng lớn nhất của nhánh này: không có chỗ cho IMU

Có một điều mà không bảng kết quả nào nói ra, và với bạn nó có thể là điều quan trọng nhất trong cả mục: *cả nhánh này thuần thị giác*. DUST3R, MAST3R và VGGT đều nhận vào ảnh và chỉ ảnh.

Hệ quả không phải là “thiếu một tính năng”. Nó là một mâu thuẫn ở gốc. Nhánh này thừa hưởng trọn vẹn tính không xác định được của tỉ lệ — đầu ra của chúng không có đơn vị mét, và mục trước đã chỉ ra rằng không lượng dữ liệu nào chứa được điều đó. Đồng thời chúng không có chỗ nào để nhận cảm biến duy nhất chứa được nó. Một hệ SLAM cổ điển chỉ cần thêm một gia tốc kế là mất hẳn bậc tự do tỉ lệ; một mô hình nền hình học không có cơ chế tương đương.

Điều đó dẫn tới ba việc phải làm nếu bạn muốn ghép nhánh này vào hệ của mình.

- **Tỉ lệ phải được cố định bên ngoài mạng.** Cách sạch nhất là để mạng lo hình dạng, còn tỉ lệ do phần cổ điển của bạn quyết, giống như phân công giữa tiên nghiệm và phép đo ở mục trước.
- **Không có bộ dự đoán chuyển động.** Mạng xử lý một nhóm ảnh như một tập, không như một chuỗi có động học. Nó không biết rằng khung sau phải gần khung trước, trong khi tích phân trước của IMU biết điều đó rất chắc.
- **Không có mốc thời gian.** Không có chỗ khai báo độ trễ giữa ảnh và IMU, và không có chỗ khai báo màn trập cuộn. Với chuyển động nhanh, cả hai thứ đó đều là nguồn sai số bậc nhất.

Nói cách khác, ba lỗ hổng ấy đúng là ba thứ mà một hệ thị giác-quán tính đã giải xong từ lâu. Đây là lý do mạnh nhất để đọc nhánh này như một *nguồn ý tưởng* và như một công cụ cho các tác vụ ngoài đường tới hạn, chứ không như một ứng viên thay thế.

↪ **tiền đề** F/34 — mọi thứ chương 34 nói về trọng số IMU đều giả định có một chỗ trong bài toán để đặt phép đo IMU vào; nhánh này chưa có chỗ đó.

G. Đọc bằng chứng của nhánh này cho đúng

Ba điều mà các bảng kết quả trong nhánh này thường không nói ra, và cả ba đều quan trọng với bạn.

- **Chất lượng ảnh dựng lại không phải độ chính xác quỹ đạo.** Hai đại lượng này bị trình bày cạnh nhau đến mức người đọc tự động gộp chúng. Một bản đồ dày đẹp mắt vẫn có thể đi kèm một quỹ đạo trôi, vì phần lớn sai số quỹ đạo là sai số tần số thấp mà mắt không thấy trên ảnh dựng lại.
- **Phép căn chỉnh trước khi đo che mất đúng thứ bạn quan tâm.** Với hệ đơn mắt, quỹ đạo phải được căn chỉnh cả tỉ lệ trước khi tính sai số, và phép căn chỉnh đó hấp thụ trọn vẹn phần trôi tỉ lệ. Hệ stereo-inertial của bạn không được hưởng ưu ái ấy, nên hai con số không so trực tiếp được với nhau.

- **Điều kiện chạy khác hẳn điều kiện của bạn.** Phần lớn kết quả đến từ chuỗi ngắn, cảnh trong nhà, và máy để bàn có GPU rời. Ba khác biệt đó cộng lại đủ để đảo ngược một kết luận, và không cái nào trong ba xuất hiện dưới dạng một cột trong bảng.

Cách đọc thực dụng: khi gặp một hệ trong nhánh này, hãy tìm ba con số trước khi tìm bảng chính — độ dài chuỗi tính bằng khung hình, phần cứng, và quỹ đạo được căn chỉnh theo phép biến đổi nào. Nếu ba con số ấy không có trong bài, phần còn lại chưa dùng để quyết định được.

→ cùng nguyên lý với E/23 — đây là cùng danh sách câu hỏi bạn dùng để đọc mọi bảng kết quả, chỉ đổi ba mục cho đúng đặc thù của nhánh này.

1.5 Cổng đánh giá: khi nào nó cứu, khi nào nó phá (The Evaluation Gate)

A. Bốn chỗ nó thật sự cứu

Gộp lại từ ba mục trên, danh sách các chỗ mà một tiên nghiệm học được thêm *thông tin* chứ không chỉ thêm số:

- **Vùng thị sai đã cạn.** Xa hơn khoảng cách mà độ chênh lệch tụt xuống dưới sàn nhiều, stereo của bạn không còn nói gì. Ở đó một phỏng đoán có sai số vừa phải vẫn tốt hơn một phép đo mà sai số đã lớn hơn nhiều lần — và điểm cắt giữa hai đường cong ấy là thứ bạn tự đo, không phải thứ chép từ bảng của ai.
- **Vùng không có vân.** Không có ghép cặp thì không có phép đo ở bất kỳ khoảng cách nào. Đây là chỗ duy nhất mà tiên nghiệm không cạnh tranh với cái gì cả.
- **Chuyển động nhanh và ảnh mờ.** Dòng quang dày cho tương ứng ở đúng dải mà bộ phát hiện góc đã tắt. Đây là chỗ có bằng chứng cơ chế mạnh nhất trong cả chương.
- **Lắp lỗ cho bản đồ dày, ở hạ nguồn.** Nếu sản phẩm cuối là bản đồ chiếm dụng để điều hướng chứ không phải quỹ đạo, thì tiên nghiệm dày là đúng công cụ — miễn là nó không quay ngược vào đồ thị tư thế.

Có một quan sát về chính tập dữ liệu của bạn đáng ghi ra, vì nó thu hẹp cổng thứ nhất đi nhiều. Các chuỗi phòng Vicon của EuRoC diễn ra trong một không gian cỡ tám mét, nên gần như không có điểm nào rơi vào dải thị sai cạn. Các chuỗi nhà máy thì rộng cỡ vài chục mét, nên ở đó cổng mới thật sự hé mở. Nghĩa là: nếu bạn đo lợi ích của độ sâu học được trên các chuỗi phòng Vicon rồi kết luận “không có tác dụng”, kết luận ấy đúng nhưng nó chỉ đúng cho những chuỗi đó. Ngược lại, một kết quả tốt trên chuỗi nhà máy cũng chưa chuyển được sang cảnh ngoài trời rộng hơn nữa. Đây là lý do mini project của chương yêu cầu hai chuỗi có quy mô không gian khác nhau, chứ không phải hai chuỗi có độ khó khác nhau.

Còn khởi tạo đơn ảnh, thứ được nhắc nhiều nhất trong tài liệu, gần như không liên quan tới bạn. Hệ stereo có đường cơ sở nên nó khởi tạo được ngay cả khi đứng yên. Phải nói thẳng điều này vì nó là nguồn lãng phí thời gian phổ biến: rất nhiều lợi ích được quảng cáo là lợi ích *của hệ đơn mắt*, và chúng biến mất khi bạn có hai camera.

B. Bốn chỗ nó phá — và phép tính chứng minh chỗ thứ nhất

Thứ nhất: nó đưa sai lệch có hệ thống vào bộ tối ưu, và đặt trọng số nhỏ không cứu được.

Đây là chỗ đáng tính bằng số nhất trong cả chương, vì kết quả đi ngược trực giác. Xét việc ước lượng một đại lượng vô hướng, chẳng hạn khoảng cách tới một mảng tường. Bạn có hai nguồn.

- 20 phép đo stereo thật, mỗi phép đo độ lệch chuẩn $\sigma_m = 0,10$ m. Thông tin đóng góp: $20/0,10^2 = 2000$.
- 5000 pixel độ sâu học được, bạn khai báo độ lệch chuẩn $\sigma_p = 0,30$ m — đã cẩn thận đặt gấp ba lần phép đo thật. Thông tin đóng góp: $5000/0,30^2 \approx 55600$.

Tiên nghiệm áp đảo phép đo gấp gần ba mươi lần. Giả sử mạng lệch đều $0,25$ m — cả mảng tường bị đẩy ra xa cùng một khoảng. Nghiệm bình phương tối thiểu có trọng số dịch đi

$$\frac{55600}{55600 + 2000} \times 0,25 \approx 0,241 \text{ m.}$$

Bạn vừa dịch nghiệm 24 cm bằng một nguồn mà bạn tưởng đã hạ trọng số.

Bây giờ thử hạ trọng số thật mạnh, khai báo $\sigma_p = 3,0$ m, tức gấp mười lần. Thông tin còn $5000/9 \approx 556$, và độ dịch còn $556/2556 \times 0,25 \approx 5,4$ cm. Muốn tiên nghiệm thật sự không đáng kể, bạn cần $5000/\sigma_p^2 \ll 2000$, tức $\sigma_p \gg 1,6$ m — một mức bất định mà ở đó tiên nghiệm chẳng còn nói gì nữa.

Vì sao lại thế? Vì bạn đang đếm sai số mẫu độc lập. Nếu 5000 pixel đó có sai số *độc lập* thì trung bình của chúng có sai số chuẩn $0,30/\sqrt{5000} \approx 0,004$ m, tức chính xác hơn phép đo stereo cả bậc độ lớn — và điều đó rõ ràng vô lý. Sự thật là sai số của chúng *tương quan gần như hoàn toàn*: cả mảng tường lệch cùng một hướng vì mạng hiểu sai cả mảng tường. Số mẫu độc lập hiệu dụng gần với 1 hơn là 5000. Một ma trận thông tin chéo giả định độc lập, nên nó thổi phồng lượng thông tin lên cỡ ba bậc độ lớn.

Suy ra cách chữa đúng, và nó không phải hạ trọng số: hãy **mô hình hoá phần tương quan bằng một biến phụ**. Thêm vào bài toán một cặp tham số tỉ lệ và độ dịch cho mỗi khung hình, để thành phần sai số chung bị hấp thụ vào tham số đó thay vì đẩy vào tư thế. Đây đúng là cách bạn đã xử lý độ lệch của con quay hồi chuyển — không hạ trọng số phép đo con quay, mà thêm một biến độ lệch vào trạng thái.

→ cùng nguyên lý với F/34 — thêm biến phụ để hấp thụ thành phần sai số chung chính là cách xử lý độ lệch cảm biến trong đồ thị nhân tử; ở đây đối tượng đối từ con quay sang mạng độ sâu.

Và đó cũng chính là bài toán mà Align3R đang tấn công ở phía mô hình.

Thứ hai: nó phá tính nhất quán đa khung. Với họ MiDaS, cặp tỉ lệ và độ dịch là của riêng từng khung hình. Một bức tường đứng yên vì thế thở phập phồng qua các khung. Bộ tối ưu của bạn tin rằng cảnh cứng, nên nó không có chỗ nào để nhét nhịp thở đó, ngoài tư thế và vận tốc. Chuỗi hậu quả rất cụ thể: tỉ lệ dao động ở tần số khung hình, vận tốc thị giác nhiễu theo, và bộ ước lượng độ lệch gia tốc kế — vốn là thứ hấp thụ mọi thứ không giải thích được giữa thị giác và IMU — bắt đầu trôi. Nói cách khác, một tiên nghiệm độ sâu đặt sai chỗ hiện ra dưới dạng một bệnh của IMU. Đó là kiểu triệu chứng tốn nhiều ngày nhất để lần ra.

Thứ ba: nó làm hỏng chẩn đoán. Ba dạng, tăng dần độ khó chịu.

- **Mất ranh giới ablation.** Một mạng là một khối. Bạn không tắt được nửa nó để xem nửa kia đóng góp gì.
- **Hỏng lặng lẽ.** Không có tín hiệu nào tương đương với “không còn điểm nội biên nào”. Mạng luôn trả về một bản đồ độ sâu trông hợp lý, kể cả trên mặt gương, kể cả khi ống kính bị bẩn.
- **Mất tính lặp lại.** Một mô hình dựa trên khuếch tán như Marigold cho hai kết quả khác nhau ở hai lần chạy trên cùng một ảnh, vì nó bắt đầu từ nhiễu ngẫu nhiên. Giả sử bạn đã bỏ công làm cho hệ của mình tắt định: ghim thứ tự duyệt container, xoá tranh chấp luồng. Cắm một khối như thế vào là tự tay dựng lại đúng cái sản phẩm bạn vừa đẹp. Và lần này không có bug nào để sửa.

Thứ tư: nó lệch miền một cách âm thầm. Metric3D cho độ sâu tuyến tính theo tiêu cự bạn khai báo, nên một sai số hiệu chỉnh 3% thành sai số độ sâu 3% đều khắp. Các mô hình huấn luyện

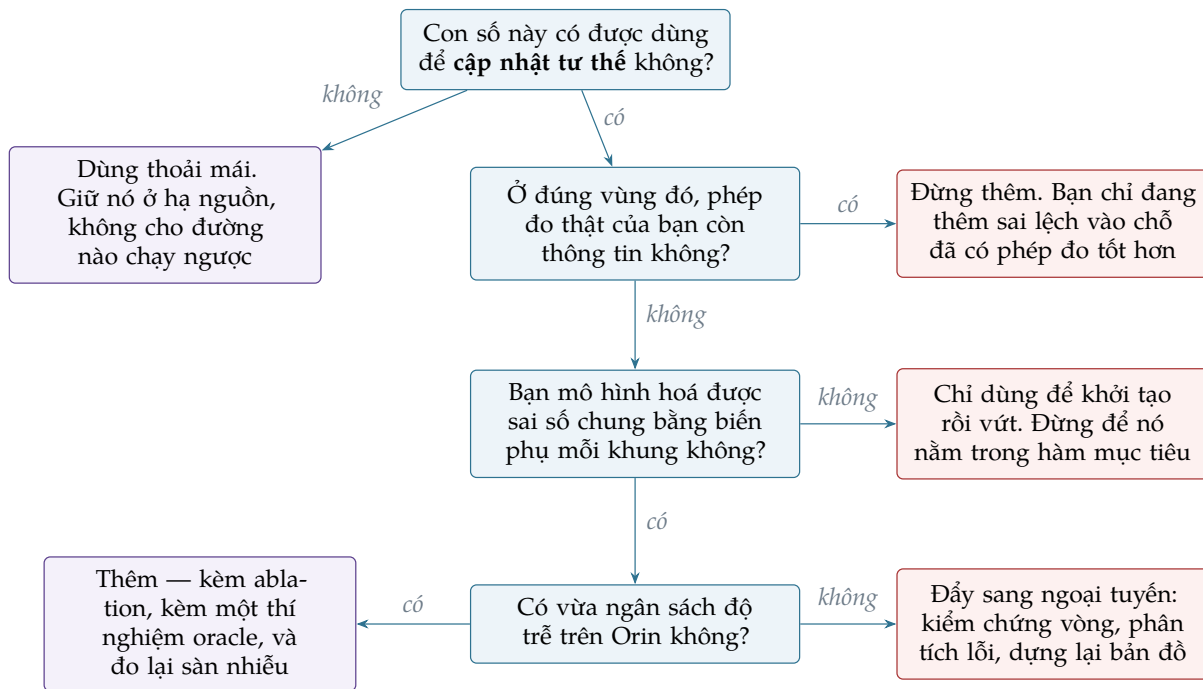
chủ yếu trên cảnh đường phố và cảnh trong nhà thông thường sẽ gặp gì ở một hành lang tối của TUM-VI, ở một nhà máy có kết cấu kim loại lặp lại, hay qua một ống kính mắt cá? Không ai biết trước, và triệu chứng duy nhất là quỹ đạo hơi tệ đi.

Bảng 1.4. Bốn chỗ cứu và bốn chỗ phá, đặt cạnh nhau. Cột cuối là điều bạn phải làm được *trước* khi tích hợp; nếu chưa làm được thì cổng chưa mở.

	Tình huống	Cơ chế	Phép kiểm tra trước khi tích hợp
Cứu	Thị sai đã cạn	ΔZ tăng theo Z^2 còn sai số tương đối của mạng thì không	Vẽ biểu đồ tỉ lệ điểm bản đồ theo khoảng cách; đo phần nằm ngoài ngưỡng một pixel độ chênh lệch
Cứu	Vùng không có vân	Không có ghép cặp thì không có phép đo để cạnh tranh	Đếm số khung hình có số điểm khớp dưới ngưỡng phục hồi
Cứu	Chuyển động nhanh, ảnh mờ	Ghép cặp thừa là chuỗi cổng nhân nhau, dòng quang không có cổng nào	Vẽ tỉ lệ điểm tìm lại được theo tốc độ góc lấy từ IMU
Cứu	Bản đồ dày ở hạ nguồn	Không phản hồi vào đồ thị tư thế nên không đưa sai lệch vào	Kiểm tra bằng mã rằng không có đường dữ liệu nào chạy ngược
Phá	Sai lệch có hệ thống	Sai số tương quan bị ma trận thông tin chéo đếm thành hàng nghìn phiếu độc lập	Tính tỉ số thông tin giữa hai nguồn trước khi viết dòng code nào
Phá	Mất nhất quán đa khung	Tỉ lệ và độ dịch riêng từng khung bị hấp thụ vào tư thế và vận tốc	Vẽ tỉ lệ ước lượng theo thời gian; nếu nó dao động ở tần số khung hình thì hỏng
Phá	Mất chẩn đoán	Không ablate được từng phần; hỏng không phát tín hiệu; đầu ra có thể ngẫu nhiên	Chạy cùng cấu hình mười lần và so sánh nhiều trước và sau khi thêm
Phá	Lệch miền	Đầu ra tuyến tính theo tiêu cự khai báo; miền huấn luyện không phủ cảnh của bạn	Chạy trên chuỗi khó nhất của bạn và so với chân trị độ sâu, không chỉ so quỹ đạo

C. Cây quyết định

Gộp tất cả lại thành một thứ dùng được trong ba mươi giây. Hình 1.9 là cây quyết định cho câu hỏi “có nên cấm tiên nghiệm học được này vào không”.



Hình 1.9. Cây quyết định cho việc cấm một tiên nghiệm dày học được vào hệ SLAM. Câu hỏi đầu tiên không phải “mô hình nào tốt nhất” mà “con số này có chạy ngược vào tư thế không”: mọi rắc rối trong chương này chỉ xuất hiện khi câu trả lời là có. Hai nhánh giữa lần lượt lọc theo thông tin (đừng cạnh tranh với phép đo tốt hơn) và theo khả năng mô hình hoá sai số tương quan. Nhánh cuối là ngân sách, và lời đáp “không” ở đó không phải lời từ chối — nó chỉ đổi chỗ dùng sang các tác vụ ngoại tuyến.

D. Bốn bài học thiết kế mang đi được, kể cả khi bạn không dùng mô hình nào

Phần này trả lời câu hỏi cuối trong bốn câu mà mọi phương pháp trong Phần F phải trả lời: nếu không dùng nguyên phương pháp thì học được gì từ thiết kế của nó. Bốn bài học dưới đây dùng lại được ngay trong mã C++ của bạn.

- **Tách phần tính một lần khỏi phần lặp** — từ *RAFT*. Bất cứ khi nào một vòng lặp tối ưu tính lại cùng một đại lượng ở mọi vòng, hãy hỏi xem đại lượng ấy có phụ thuộc nghiệm hiện tại không. Nếu không, đưa nó ra ngoài và biến vòng lặp thành phép tra cứu. Hệ quả kèm theo cũng quý: số vòng lặp trở thành núm vặn độ trễ chỉnh được lúc chạy.
- **Làm thô đúng trục** — cũng từ *RAFT*. Mọi sơ đồ đa tỉ lệ đều phải chọn làm thô cái gì. Làm thô sai trục thì bạn mất đúng thứ mình đang tìm, và mọi mức mịn hơn chỉ tinh chỉnh quanh chỗ sai. Câu hỏi cần đặt mỗi lần thiết kế một kim tự tháp: *trục nào ở đây không được phép mất độ phân giải*.
- **Ẩn số mà dữ liệu không nói tới thì phải ghim, không phải học** — từ *Metric3D*. Đưa nó ra khỏi phần học, chuẩn hoá ở đầu vào, khôi phục ở đầu ra. Đây là cổ định gauge, và bạn đã dùng nó mỗi lần cố định khung khoá đầu.
- **Muốn có trọng số dùng được thì phải huấn luyện cho nó** — từ *SEA-RAFT*. Một hàm mất mát đơn phương thức không thể diễn đạt “câu trả lời ở đây có hai đáp án”. Nếu bạn cần một hiệp phương sai để đưa vào đồ thị nhân tử thì phải chọn hàm mất mát cho phép biểu diễn nó, ngay từ lúc huấn luyện.
- **Lớp bất định của đầu ra quyết định đa tạp của bộ tối ưu** — từ *VGGT-SLAM*. Trước khi viết bộ ghép mảnh bản đồ, hãy hỏi hai mảnh ấy thật sự liên hệ với nhau bằng nhóm biến đổi nào. Chọn nhầm nhóm là sai mô hình, và sai mô hình không hiện ra dưới dạng hội tụ chậm — nó hiện ra dưới dạng một nghiệm sai mà bộ tối ưu rất tự tin.

Bài học thứ sáu không thuộc riêng công trình nào mà thuộc cả chương, và nó là bài học đắt nhất: *đổi biểu diễn đầu ra xoá được nhiều bước hơn là đổi kiến trúc*. DUS3R không giỏi hơn pipeline cổ điển ở bất kỳ bước riêng lẻ nào; nó chỉ chọn một đầu ra làm cho phần lớn các bước trở nên không cần thiết.

Bảng 1.5. Ba họ, đọc theo *cấu trúc* chi phí thay vì theo con số. Bảng này cố ý không có cột mili-giây hay cột megabyte: một con số như thế chỉ đúng cho một bản cài đặt, một trình biên dịch suy luận và một mức tải cụ thể, nên chép nó vào tài liệu học là dạy sai. Cột giữa nói chi phí *tỉ lệ với cái gì* — đó mới là thứ không đổi — và cột phải nói phép đo bạn phải chạy để biến nó thành một con số dùng được.

Họ	Chi phí tỉ lệ với cái gì	Chỗ dùng được	Phép đo phải chạy
Độ sâu đơn ảnh, bản nhỏ	Một lượt xuôi mỗi lần gọi; gọi được ở tần số thấp hơn tần số khung hình	Có thể vừa ngân sách nếu bạn chấp nhận cập nhật thưa hơn ảnh	Biên dịch bằng trình suy luận của thiết bị, đo độ trễ khi hệ đang chạy đủ tải
Độ sâu khuếch tán	Số bước khử nhiễu nhân một lượt xuôi, và phải chạy nhiều lần rồi lấy trung bình	Chỉ ngoại tuyến; thêm nữa nó không tắt định	Chạy cùng ảnh nhiều lần và đo độ tàn mạn của đầu ra
Dòng quang dày	Bình phương độ phân giải lưới cho khối tương quan, cộng số vòng lặp	Ngang cả ngân sách <i>frontend</i> hiện tại, nên là quyết định thay thế chứ không phải thêm vào	Đo bộ nhớ đỉnh và độ trễ ở đúng độ phân giải bạn định dùng
Mô hình nền hình học	Bậc hai theo số khung (nhánh theo cặp) hoặc tuyến tính dốc theo số token (nhánh toàn cục)	Chỉ dùng ngoài đường tới hạn: kiểm chứng vòng, dựng lại bản đồ, phân tích lỗi	Tăng dần số khung cho tới khi hết bộ nhớ; đó là trần thật của bạn

CẦN NHỚ

Ba họ mô hình trong chương này không cho bạn phép đo mới; chúng cho bạn phỏng đoán dày. Với một hệ đã có tỉ lệ mét thật, giá trị của chúng chỉ nằm ở đúng những chỗ phép đo thật đã cạn — thị sai dưới sàn nhiều, vùng không có vân, khung bị mờ. Ở mọi chỗ khác, chúng chỉ thêm sai lệch có hệ thống. Và cái giá cố định, không thương lượng được, là: *chúng không bao giờ nói “tôi không biết”*. Hệ cổ điển của bạn hồng âm ỉ bằng cách trả về không điểm nội biên; một mạng hồng lặng lẽ bằng cách trả về một câu trả lời trông rất hợp lý.

1.6 Giới hạn và trường hợp hồng

Mục này nói về giới hạn của chính chương, không chỉ của các phương pháp trong chương.

Chương này cố ý không đưa con số ngân sách nào. Không có mili-giây trên Orin, không có megabyte, không có FLOP. Lý do không phải là lười: một ước lượng tự tính từ đếm token hay đếm phép nhân cộng có sai số cả bậc độ lớn so với thực tế, vì nó bỏ qua hiệu suất sử dụng, trình biên dịch suy luận, và các thủ thuật tiết kiệm trong bản cài đặt thật — nhưng khi đã in ra giấy thì nó *trông* như một phép đo và bị dùng như một phép đo. Thứ chương này đưa thay vào đó là *dạng* của quan hệ: chi phí tỉ lệ với cái gì, và tăng theo số mũ nào. Dạng ấy không đổi theo phần cứng, nên nó dùng được lâu; con số thì bạn phải tự đo, và Bảng 1.5 nói rõ mỗi họ đo bằng cách nào.

Lập luận về ảnh mờ là lập luận cơ chế, không phải đường cong đo được. Chuyện giá trị riêng của ma trận mô men bậc hai giảm cỡ L lần là đúng về bậc, nhưng bộ phát hiện thật có ngưỡng phi tuyến và có bước triệt phi cực đại, nên đường cong thật có thể dốc hơn hoặc thoải hơn nhiều. Đó là lý do mini project thứ hai của chương đi đo chính đường cong này.

Phép tính về sai lệch có hệ thống đã đơn giản hoá bài toán xuống một chiều. Trong bài toán tư thế đầy đủ, hướng mà sai lệch đẩy nghiệm phụ thuộc vào hình học của cảnh, và một phần của nó có thể bị chiếu triệt tiêu. Hãy coi con số 24 cm là chặn trên trên một trục, không phải dự báo cho ATE.

Ba giả định chung của cả ba họ, và chúng vỡ ở đâu.

- **Màn trập toàn cục.** Mọi mô hình trong chương này giả định một khung hình ứng với một thời điểm. Camera màn trập cuộn dưới chuyển động nhanh vì phạm vi điều đó ngay ở mức pixel, và một bản đồ điểm sẽ pha trộn hình học của nhiều thời điểm mà không có chỗ nào để khai báo điều đó.
- **Cảnh cứng.** DUST3R, VGGT và phần lớn dữ liệu huấn luyện của chúng là cảnh tĩnh. Người đi qua trước ống kính không chỉ làm sai vùng đó; nó làm hỏng cả phép căn chỉnh vì bản đồ điểm được hồi quy chung.
- **Gần lỗ kim.** Metric3D chuẩn hoá theo một tiêu cự, và DUST3R suy nội tham số từ bản đồ điểm với giả định điểm chính nằm giữa ảnh. Ống kính mắt cá hoặc méo mạnh phá cả hai, và triệu chứng là sai số biến thiên theo bán kính ảnh chứ không đều.

Ngân sách GPU là ràng buộc cứng nằm ngoài chương. Nếu GPU của Orin đã bận cho một khối khác — một bộ phát hiện vật thể, một bộ khử nhiễu — thì mọi lựa chọn trong chương này đóng lại, bất kể chúng chính xác tới đâu. Cổng ngân sách phải mở trước cổng độ chính xác.

Các công trình 2025–2026 được mô tả theo phương pháp, không theo kết quả tôi tự kiểm chứng. MAST3R-SLAM, VGGT, VGGT-SLAM và bản thứ ba của Depth Anything đều rất mới. Cơ chế mà tôi trình bày lấy từ mô tả phương pháp và tôi tin là đúng; các nhận định về hiệu năng thì tôi cố ý không đưa ra. Trần bộ nhớ và trần số khung cũng là thứ sẽ dịch chuyển nhanh nhất khi có bản cài đặt tốt hơn.

1.7 Thực hành

Repo và tài nguyên. Độ sâu đơn ảnh: [isl-org/MiDaS](https://github.com/isl-org/MiDaS) là bản tham chiếu để đọc hàm mất mát bất biến tỉ lệ và dịch — hãy mở thẳng phần cài đặt phép căn bậc nhất, nó ngắn và nói rõ hơn cả bài báo. [DepthAnything/Depth-Anything-V2](https://github.com/DepthAnything/Depth-Anything-V2) cho bản dùng được nhất hiện nay, [YvanYin/Metric3D](https://github.com/YvanYin/Metric3D) cho phần chuẩn hoá theo tiêu cự, [prs-eth/Marigold](https://github.com/prs-eth/Marigold) nếu muốn thấy cách một mô hình khuếch tán được tái dụng. Dòng quang: [princeton-vl/RAFT](https://github.com/princeton-vl/RAFT) rồi [princeton-vl/SEA-RAFT](https://github.com/princeton-vl/SEA-RAFT); đọc RAFT trước vì bản gốc gọn hơn, và hàm tra cứu là chừng bốn mươi dòng đáng đọc từng dòng. Mô hình nền hình học: [naver/dust3r](https://github.com/naver/dust3r), [naver/mast3r](https://github.com/naver/mast3r), [facebookresearch/vgggt](https://github.com/facebookresearch/vgggt), và [rmurai0610/MASt3R-SLAM](https://github.com/rmurai0610/MASt3R-SLAM) nếu muốn xem một hệ SLAM hoàn chỉnh dựng trên chúng. Đối chiếu với hệ cổ điển thì vẫn là [COLMAP](https://github.com/colmap/colmap) và [evo](https://github.com/ethz-asl/evo). Danh sách tên mô hình này chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; hạ tầng như COLMAP, evo và trình biên dịch suy luận thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Với cấu hình stereo của bạn, ở khoảng cách nào thì độ bất định độ sâu vượt 10% — và bạn tính ra nó bằng công thức nào? Nếu một mô hình họ MiDaS được căn bằng đúng một hằng số nhân, chân trời cứng của nó nằm ở đâu và vì sao nó tồn tại? Vì sao việc gộp trung bình khối tương quan chỉ theo hai chiều của ảnh thứ hai lại là chi tiết quyết định của RAFT? Nếu DUST3R không ước lượng tư thế thì tư thế nằm ở chỗ nào trong đầu ra của nó, và bạn đọc nó ra bằng phép toán gì?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●● MonoDepth	Mạng đoán độ chênh lệch từ một ảnh, huấn luyện bằng sai lệch quang trắc sau khi làm cong ảnh stereo còn lại. Vì đường cơ sở lúc huấn luyện đã biết nên đầu ra có đơn vị mét — nhưng chỉ đúng cho đúng cấu hình camera đó.
●●● Độ sâu bất biến tỉ lệ và dịch	Đầu ra chỉ xác định sai khác một phép biến đổi bậc nhất trên độ sâu nghịch đảo: $d \approx s/Z + t$. Mạng biết thứ tự và tỉ lệ tương đối, không biết đơn vị và không biết gốc. Hai ẩn đó khác nhau ở mỗi khung hình.
●●● MiDaS	Trộn nhiều tập dữ liệu có nhãn không cùng đơn vị, bằng cách căn dự đoán về nhãn qua một phép biến đổi bậc nhất tối ưu trước khi tính sai lệch. Đối lấy khả năng khái quát rất tốt, cái giá là mất hẳn đơn vị mét.
●●● Metric3D	Lấy lại mét bằng cách chuẩn hoá mọi ảnh về một camera chuẩn theo tiêu cự, dự đoán, rồi nhân ngược lại. Đây là cố định gauge: một ẩn số dữ liệu không nói gì tới thì bị ghim ở bước tiền xử lý. Hệ quả: sai 3% ở tiêu cự thành sai 3% ở độ sâu.
●● Depth Anything V2	Thầy huấn luyện hoàn toàn trên ảnh tổng hợp để nhận hoàn hảo, rồi gán nhãn giả cho hàng chục triệu ảnh thật để lấp khoảng cách miền. Bài học: mở rộng dữ liệu, không mở rộng kiến trúc.
●●● Độ sâu quy ra mét bảo hoà ở một khoảng cách cố định — nguyên nhân?	Bạn cần bằng một hằng số nhân duy nhất, tức ngầm đặt độ dịch bằng không. Khi đó $\hat{Z} = Z/(1 + tZ/s)$ và nó bảo hoà ở s/t . Phân biệt với lệch miền: chân trời cứng cho sai số tăng đơn điệu theo khoảng cách, lệch miền thì không có dạng đó.
●●● Chân trời stereo	Khoảng cách mà độ chênh lệch tụt xuống dưới sàn nhiễu. Từ $Z = fb/d$ suy ra $\Delta Z = Z^2 \Delta d/(fb)$: sai số tăng theo bình phương khoảng cách. Với $fb \approx 50$ của EuRoC, độ chênh lệch một pixel đã ứng với 50 m.
●●● Khối tương quan toàn cặp	Tích vô hướng đặc trưng giữa mọi vị trí của ảnh một và mọi vị trí của ảnh hai, tính một lần. Với lưới 1/8 của ảnh EuRoC là 5640^2 phần tử — chi phí bộ nhớ tăng theo bình phương độ phân giải lưới, nên hạ độ phân giải là núm vặn mạnh nhất.
●●● RAFT	Tách “chỗ nào giống chỗ nào” — tính một lần — khỏi “tôi đang đoán ở đâu” — tra cứu lặp quanh phỏng đoán hiện tại, cập nhật bằng một khối hồi tiếp. Số vòng lặp là núm vặn độ trễ chỉnh được lúc chạy.
●● Vì sao RAFT không cần thu nhỏ ảnh?	Vì kim tự tháp của nó dựng trên <i>khối tương quan</i> , và chỉ gộp hai chiều thuộc ảnh thứ hai. Lưới của ảnh một giữ nguyên độ phân giải ở mọi mức, nên vật nhỏ bay nhanh không biến mất — đúng chỗ chiến lược thô tới mịn thất bại.
●● SEA-RAFT	Bản RAFT gọn hơn: hồi quy thẳng nghiệm khởi đầu nên cần ít vòng lặp hơn, và đổi hàm mất mát sang hỗn hợp hai phân phối Laplace. Nhờ hàm mất mát đó nó xuất ra được độ bất định theo từng pixel, dùng thẳng làm trọng số.
●●● Vì sao ghép cặp thưa sụp đổ dưới ảnh mờ còn dòng quang chỉ xuống dốc?	Vì ghép cặp thưa là chuỗi cổng nhân nhau: phải phát hiện được ở cả hai khung, rồi mới tới mô tả và kiểm tra tỉ số. Tính lặp lại tụt từ 0,8 xuống 0,3 thì xác suất sống qua cả hai khung tụt từ 0,64 xuống 0,09. Dòng quang không có cổng phát hiện nào.
●●● Bản đồ điểm (pointmap)	Tensor $H \times W \times 3$ gán thẳng cho mỗi pixel một điểm ba chiều trong một hệ toạ độ cho trước. Khác bản đồ độ sâu ở chỗ nó đã mang sẵn hệ toạ độ, nên đọc ra hình học mà không cần biết tiêu cự trước.
●●● DUST3R	Nhận hai ảnh chưa hiệu chỉnh, trả về hai bản đồ điểm <i>cùng nằm trong hệ toạ độ của camera một</i> . Tư thế không biến mất mà bị gói vào đầu ra, nên nó đọc ra bằng Procrustes chứ không cần ma trận cơ bản. Đầu ra không có đơn vị mét.
●● MAST3R	DUST3R cộng một đầu ra mô tả đặc trưng dày và một thủ tục ghép tương hỗ nhanh. Kết quả là bộ ghép cặp neo trong ba chiều, nên nó chịu được thay đổi góc nhìn cực đoan — chính chỗ mà mọi bộ mô tả hai chiều chết.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• VGGT	Một <i>transformer</i> nhận nhiều ảnh cùng lúc, xen kẽ attention trong từng ảnh và attention trên toàn bộ token của mọi ảnh, rồi trả thẳng ra tư thế, độ sâu và bản đồ điểm. Không còn bước căn chỉnh toàn cục; đổi lại bộ nhớ attention toàn cục đặt trần cứng lên số khung.
• VGGT-SLAM	Ghép các mảnh bản đồ của VGGT trên nhóm xạ ảnh SL(4) thay vì phép biến đổi tương tự, vì với camera chưa hiệu chỉnh thì lớp bất định là xạ ảnh. Bài học: lớp bất định của frontend quyết định đa tạp mà backend phải tối ưu trên đó.
••• Vì sao hạ trọng số không cấu được một tiên nghiệm dày?	Vì sai số của nó tương quan gần như hoàn toàn, còn ma trận thông tin chéo lại đếm 5000 pixel thành 5000 phiếu độc lập. Với 20 phép đo $\sigma = 0,1$ và 5000 tiên nghiệm $\sigma = 0,3$, tiên nghiệm áp đảo gần ba mươi lần. Cách chữa đúng là thêm biến phụ mỗi khung để hấp thụ sai số chung.
•• Bản đồ dày thờ phập phồng và độ lệch gia tốc kể trôi — nghĩa là gì?	Bạn đang nhét độ sâu bất biến tỉ lệ và dịch vào một bộ tối ưu tin rằng cảnh cứng. Tỉ lệ riêng từng khung bị hấp thụ vào tư thế và vận tốc, và độ lệch gia tốc kể — vốn hấp thụ mọi thứ không giải thích được — lãnh phần còn lại.
•• Cùng một ảnh, hai lần chạy cho hai bản đồ độ sâu khác nhau — nguyên nhân?	Mô hình dựa trên khuếch tán như Marigold khởi đầu từ nhiễu ngẫu nhiên, nên nó vốn ngẫu nhiên; người ta phải chạy nhiều lần rồi lấy trung bình. Phân biệt với phi tất định của thư viện: cố định hạt giống mà vẫn khác thì là thư viện, cố định hạt giống mà giống nhau thì là bản chất mô hình.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DROID-SLAM [22]	Dùng nguyên vòng lặp tra cứu trên khối tương quan của RAFT làm <i>frontend</i> , rồi nối vào một tầng hiệu chỉnh chùm tia khả vi	Chi tiết ở F/31; đây là bằng chứng rõ nhất rằng ý tưởng của RAFT sống ngoài bài toán dòng quang
MASt3R-SLAM [19]	Đặt thẳng một hệ SLAM dày lên tiên nghiệm bản đồ điểm hai ảnh, và tối ưu trong không gian bản đồ điểm	Công trình 2025; nêu cơ chế, không nêu số
Depth-Anything-V2 [8]	Công thức thầy tổng hợp cộng nhân giả trên ảnh thật — mở rộng dữ liệu thay vì mở rộng kiến trúc	Được dùng rộng rãi làm nguồn độ sâu trong các pipeline dựng lại và điều hướng
Marigold [9], <i>diffusers</i>	Tái dụng mô hình khuếch tán sinh ảnh làm bộ ước lượng độ sâu, huấn luyện trên rất ít dữ liệu	Hạ tầng <i>diffusers</i> bền hơn nhiều so với tên mô hình
SEA-RAFT [15]	Hàm mất mát hỗn hợp Laplace để mô hình được tính đa phương thức và xuất ra độ bất định theo pixel	Đây là cầu nối thẳng sang bài toán hiệp phương sai học được ở F/34
VGGT [20]	Xen kẽ attention trong ảnh và attention toàn cục để bỏ hẳn bước căn chỉnh toàn cục	Công trình 2025; trần số khung là hệ quả cấu trúc của attention toàn cục, mức cụ thể phải tự do

MINI PROJECT SLAM • Đo trần đóng góp của độ sâu học được bằng một thí nghiệm oracle • 1–2 ngày

Mục tiêu trả lời câu “độ sâu học được có thêm được gì cho hệ stereo-inertial của tôi không” *trước* khi cài bất kỳ mạng nào. Nếu một mô hình hoàn hảo cũng chỉ nâng APE dưới sàn nhiễu, cả hướng này đóng lại và bạn tiết kiệm được nhiều tuần.

Dữ liệu	hai chuỗi EuRoC có độ khó khác nhau, chẳng hạn MH_01 và V2_03. Chân trị tư thế đi kèm; nếu bản tải của bạn có bản quét laser phòng Vicon thì dùng nó làm chân trị hình học, nếu không thì dùng bản đồ dày do chính hệ dựng sau khi tối ưu toàn cục làm chân trị xấp xỉ và ghi rõ hạn chế đó.
Các bước	(1) chạy hệ hiện tại, ghi độ sâu của mọi điểm bản đồ tại thời điểm tam giác hoá, vẽ biểu đồ phân bố và đếm tỉ lệ điểm có độ chênh lệch dưới 1 pixel và dưới 2 pixel — đó là phần “thị sai đã cặn”; (2) dựng bản đồ độ sâu oracle cho từng khung khoá từ chân trị; (3) thay độ sâu của riêng nhóm thị sai cặn bằng giá trị oracle, giữ nguyên mọi thứ khác, chạy năm lần mỗi cấu hình; (4) so chênh lệch APE với sàn nhiều bạn đã đo ở chương 24.
Xong khi	bạn phát biểu được một câu có số chống lưng: “trên chuỗi này, một mô hình độ sâu hoàn hảo nâng APE nhiều nhất X cm, và X lớn hơn hay nhỏ hơn sàn nhiều”. Kiểm tra chéo: tỉ lệ điểm thị sai cặn phải lớn hơn hẳn ở chuỗi ngoài trời hoặc chuỗi có cảnh xa; nếu hai chuỗi cho cùng tỉ lệ thì khả năng cao bạn đang đếm sai.
Nối với	E/24 cho thiết kế thí nghiệm oracle và cách đọc sàn nhiều; F/27 cho khung bốn cổng; kết quả của dự án này quyết định có nên đọc kỹ F/32 hay không.

MINI PROJECT SLAM · Dòng quang dày có cứu được các khung mất bám không · 1–2 ngày

Mục tiêu	biến câu “dòng quang chịu được ảnh mờ tốt hơn” thành một ngưỡng tốc độ góc cụ thể cho hệ của bạn, kèm một con số độ trễ đo trên Orin chứ không ước lượng.
Dữ liệu	các đoạn mất bám hoặc số điểm khớp thấp trong log chạy của bạn trên V2_03 và MH_05; chân trị tư thế của EuRoC; chuỗi IMU để lấy tốc độ góc.
Các bước	(1) trích danh sách các cặp khung hình có số điểm khớp dưới ngưỡng phục hồi; (2) với mỗi cặp, chạy princeton-vl/SEA-RAFT ngoại tuyến, lấy trường dòng, chiếu các điểm bản đồ của khung trước theo trường đó và đếm bao nhiêu điểm rơi đúng chỗ theo chân trị tư thế; (3) gán cho mỗi cặp một tốc độ góc trung bình từ IMU và một chỉ số độ mờ, chẳng hạn năng lượng gradient đã chuẩn hoá; (4) vẽ tỉ lệ điểm tìm lại được theo tốc độ góc cho cả hai nhánh, thưa và dày; (5) biên dịch mạng bằng trình suy luận của Jetson ở đúng độ phân giải EuRoC và đo độ trễ.
Xong khi	bạn có hai đường cong trên cùng một trục, một ngưỡng tốc độ góc mà trên đó nhánh dày mới có lãi, và một con số độ trễ đo được. Từ ba thứ đó bạn phát biểu được: “dòng quang dày đáng bật ở $X\%$ số khung hình của chuỗi này, và nó tốn Y ms mỗi lần bật”.
Nối với	F/28 cho phía đặc trưng học được của cùng bài toán; F/34 cho việc biến độ bất định theo pixel của SEA-RAFT thành trọng số thật; dự án trước cung cấp sàn nhiều để đọc kết quả.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Chỉ ở đúng những chỗ phép đo của bạn đã cặn. Với $fb \approx 50$ của EuRoC và giả định độ chính xác độ chênh lệch 0,2 pixel, sai số stereo tăng theo Z^2 và vượt mười phần trăm ở vài chục mét, trong khi sai số tương đối của mạng không tăng theo khoảng cách — nên hai đường cong bắt buộc cắt nhau, và điểm cắt là thứ bạn phải tự đo. Cộng thêm vùng không có vân, nơi bạn không có phép đo nào để cạnh tranh. Ngoài hai chỗ đó, nó chỉ thêm sai lệch có hệ thống.
- A2. Nó dựng kim tự tháp trên *khối tương quan*, và chỉ gộp trung bình hai chiều thuộc ảnh thứ hai. Lưới của ảnh một giữ nguyên độ phân giải ở mọi mức, nên vật nhỏ không bao giờ bị thu nhỏ tới mức biến mất. Chiến lược thô tới mịn cổ điển thu nhỏ chính bức ảnh, nên nó đánh mất vật nhỏ trước khi kịp ước lượng.
- A3. Tư thế nằm trong chính đầu ra. Cả hai bản đồ điểm đều được dự đoán trong hệ toạ độ của camera một. Vì thế phép biến đổi cứng khớp bản đồ điểm của ảnh hai với hình học của camera hai chính là tư thế tương đối, và nó có nghiệm dạng đóng. Bước ước lượng bị đối thành bước đọc.
- A4. Vì bạn đang chống lại một tỉ số đếm: 5000 so với 20 là lợi thế 250 lần, nên muốn triệt tiêu nó phải nhân phương sai lên 250 lần, tức nhân độ lệch chuẩn lên gần 16 lần. Với $\sigma_m = 0,1$ thì bạn

phải khai báo $\sigma_p \gg 1,6$ m, mức mà tiên nghiệm không còn nói gì. Phép tính ba mươi giây là so hai đại lượng N/σ^2 .

- A5. Vì nó khởi đầu từ nhiễu ngẫu nhiên, nên cùng một ảnh cho hai đầu ra khác nhau. Bạn vừa dựng lại một sàn nhiễu ngay giữa pipeline, và lần này không có bug nào để sửa — khác hẳn với thứ tự duyệt container hay tranh chấp luồng. Một mô hình hồi quy thì tất định theo trọng số, nên nó không mang thêm nguồn phi tất định nào.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn cần biết mọi vật trong phòng cách bạn bao xa. Bạn có hai con mắt cách nhau mười một phân, nên với thứ ở gần bạn đo được khá chuẩn: hai mắt nhìn lệch nhau, độ lệch càng lớn thì vật càng gần. Nhưng với thứ ở cuối hành lang, độ lệch giữa hai mắt gần như bằng không, và bạn hết cách. Bây giờ có một người bạn chưa từng cầm thước bao giờ, nhưng đã nhìn hàng triệu căn phòng trong đời. Hỏi anh ta ghế kia xa bao nhiêu, anh ta nói ngay, và anh ta nói cho cả nghìn điểm trong phòng cùng lúc. Anh ta luôn đúng về chuyện cái nào gần hơn cái nào. Anh ta gần như luôn sai về chuyện bao nhiêu mét, và cái sai ấy đổi mỗi khi bạn đưa cho anh ta một bức ảnh khác.

Chuyện gì xảy ra nếu bạn ghi lời anh ta vào cùng cuốn sổ với các phép đo của mình rồi lấy trung bình? Bạn đo hai mươi lần, anh ta đoán năm nghìn lần, nên trung bình sẽ nghiêng hẳn về phía anh ta. Tệ hơn: khi anh ta sai, anh ta sai cả năm nghìn lần theo cùng một hướng, nên việc có nhiều số không giúp gì cả. Và điều nguy hiểm nhất là anh ta không bao giờ nói “chỗ này tôi không nhìn rõ”. Anh ta luôn có câu trả lời, kể cả khi nhìn vào một tấm gương.

Cách dùng anh ta cho đúng vì thế rất hẹp: chỉ hỏi ở cuối hành lang, chỗ thước của bạn đã hết tầm; và ghi lời anh ta vào một trang riêng, đừng trộn vào trang có số đo.

Chỗ không rút gọn được: chuyện “cái sai đổi mỗi khi đưa một bức ảnh khác”. Mọi cách nói gọn hơn đều làm sai lệch, vì đây không phải chuyện anh ta đoán lúc trúng lúc trượt. Cái đổi là *đơn vị* anh ta đang dùng, và nó đổi âm thầm giữa hai bức ảnh chụp cách nhau một phần ba mươi giây. Một bức tường đứng yên vì thế lúc gần lúc xa, và hệ thống của bạn — vốn tin rằng tường không di chuyển — buộc phải kết luận rằng chính bạn mới là người đang nhúc nhích.

Một câu: *Chúng cho bạn hàng nghìn phỏng đoán tự tin ở chỗ bạn chỉ có vài chục phép đo thật, nên hãy chỉ hỏi chúng ở đúng chỗ phép đo của bạn đã hết.*

1.8 Kết nối

F/27-dat-hoc-sau-vao-dau là bộ lọc mà chương này áp dụng: bốn cổng — bài toán có thật, đo được, vừa ngân sách, hổng có kiểm soát — xuất hiện lại ở đây dưới dạng cây quyết định. F/28-dac-trung-va-ghep-cap là mặt đối lập trực tiếp: nó giữ cấu trúc thừa và thay chất lượng của từng điểm, còn chương này bỏ luôn cấu trúc thừa. Đọc hai chương cạnh nhau là cách nhanh nhất để thấy đánh đổi giữa “ít phép đo nhưng kiểm chứng được” và “nhiều phỏng đoán nhưng không kiểm chứng được”. F/29-dinh-vi-lai-va-dong-vong nhận từ chương này một lối thoát cụ thể cho tầng kiểm chứng hình học: ghép cặp neo trong ba chiều không cần hai mảnh ảnh trông giống nhau. F/31-backend-kha-vi-va-dau-cuoi là bước tiếp theo tự nhiên, vì DROID-SLAM chính là vòng lặp tra cứu của RAFT nối vào một tầng hiệu chỉnh chùm tia khả vi. F/32-ban-do-neural tiêu thụ đầu ra của chương này: một bản đồ dày cần một nguồn độ sâu, và chất lượng nguồn đó quyết định chất lượng bản đồ. F/34-imu-bat-dinh-va-trien-khai là nơi kết luận quan trọng nhất của chương được xử lý đàng hoàng: muốn dùng một tiên nghiệm dày thì phải có trọng số đúng, và trọng số đúng phải được huấn luyện chứ không được đoán. Về phía nền: C/16/02-bieu-dien-3d-va-depth là tiền đề hình học, C/15/03-flow-tracking-va-rang-buoc là tiền đề về dòng quang, B/11-gioi-han-tinh-toan cho khung phân tích bộ nhớ và băng thông, và E/24-thi-nghiem-va-ablation cho thí nghiệm oracle mà cả hai mini project của chương đều dựa vào.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao việc nhân toàn bộ cảnh và toàn bộ quãng đường camera với cùng một hệ số lại khiến độ sâu đơn ảnh trở thành bài toán *không giải được* chứ không phải bài toán khó, và điều đó ràng buộc gì lên mọi mô hình trong mục 2?
2. Metric3D ghim ẩn số tiêu cự ở bước tiền xử lý thay vì để mạng học. Chỗ nào trong công việc SLAM của bạn dùng đúng thủ thuật đó, và cái giá tương ứng ở cả hai chỗ là gì?
3. Nếu bạn chỉ được giữ *một* trong hai thành phần của RAFT — khối tương quan toàn cặp, hoặc vòng lặp tra cứu bằng khối hồi tiếp — thì bỏ cái nào ít hại hơn cho bài toán chuyển động nhanh, và vì sao?
4. Dòng quang dày luôn trả lời, kể cả ở vùng bị che. Vì sao tính chất đó lại nguy hiểm hơn cho một hệ SLAM so với việc bộ ghép cặp thừa trả về không điểm nội biên?
5. DUS3R xoá bước ước lượng tư thế nhưng không xoá được tính không xác định được của tỉ lệ. Vì sao một cái xoá được còn cái kia thì không?
6. Bạn hạ trọng số của một tiên nghiệm độ sâu xuống mười lần mà nghiệm vẫn dịch mấy centimet. Điều đó nói gì về cấu trúc sai số của tiên nghiệm, và cách chữa đúng là gì thay vì hạ tiếp?
7. Trần số khung của VGGT đến từ bộ nhớ attention toàn cục, còn trần của DUS3R đến từ số cặp. Hai trần đó dẫn tới hai chiến lược chia mảnh bản đồ khác nhau như thế nào?

- [1] C. Godard, O. Mac Aodha, and G. J. Brostow, “Unsupervised Monocular Depth Estimation with Left-Right Consistency,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [2] C. Godard, O. Mac Aodha, M. Firman, et al., “Digging Into Self-Supervised Monocular Depth Estimation,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [3] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [4] R. Ranftl, A. Bochkovskiy, and V. Koltun, “Vision Transformers for Dense Prediction,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [5] S. F. Bhat, R. Birkel, D. Wofk, et al., “ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth,” *arXiv preprint arXiv:2302.12288*, 2023.
- [6] W. Yin, C. Zhang, H. Chen, et al., “Metric3D: Towards Zero-shot Metric 3D Prediction from a Single Image,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] L. Yang, B. Kang, Z. Huang, et al., “Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [8] L. Yang, B. Kang, Z. Huang, et al., “Depth Anything V2,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [9] B. Ke, A. Obukhov, S. Huang, et al., “Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [10] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [11] E. Ilg, N. Mayer, T. Saikia, et al., “FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [12] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [13] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [14] Z. Huang, X. Shi, C. Zhang, et al., “FlowFormer: A Transformer Architecture for Optical Flow,” in *European Conference on Computer Vision (ECCV)*, 2022.
- [15] Y. Wang, L. Lipson, and J. Deng, “SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [16] Z. Teed and J. Deng, “RAFT-3D: Scene Flow using Rigid-Motion Embeddings,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [17] S. Wang, V. Leroy, Y. Cabon, et al., “DUS3R: Geometric 3D Vision Made Easy,” in *CVPR*, 2024.
- [18] V. Leroy, Y. Cabon, and J. Revaud, “Grounding Image Matching in 3D with MAS3R,” in *European Conference on Computer Vision (ECCV)*, 2024.

- [19] R. Murai, E. Dexheimer, and A. J. Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [20] J. Wang, M. Chen, N. Karaev, et al., “VGGT: Visual Geometry Grounded Transformer,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [21] D. Maggio, H. Lim, and L. Carlone, “VGGT-SLAM: Dense RGB SLAM Optimized on the $SL(4)$ Manifold,” *arXiv preprint arXiv:2505.12549*, 2025.
- [22] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.